



PRACTICAL EXERCISES: 15 PERIODS

1. Install Virtualbox/VMware/ Equivalent open source cloud Workstation with different flavours of Linux or Windows OS on top of windows 8 and above.
2. Install a C compiler in the virtual machine created using a virtual box and execute Simple Programs
3. Install Google App Engine. Create a hello world app and other simple web applications using python/java.
4. Use the GAE launcher to launch the web applications.
5. Simulate a cloud scenario using CloudSim and run a scheduling algorithm that is not present in CloudSim.
6. Find a procedure to transfer the files from one virtual machine to another virtual machine.
7. Install Hadoop single node cluster and run simple applications like wordcount.
8. Creating and Executing Your First Container Using Docker.
9. Run a Container from Docker Hub

Ex.No :01	**Virtual Workstation**
Date:	

Aim:

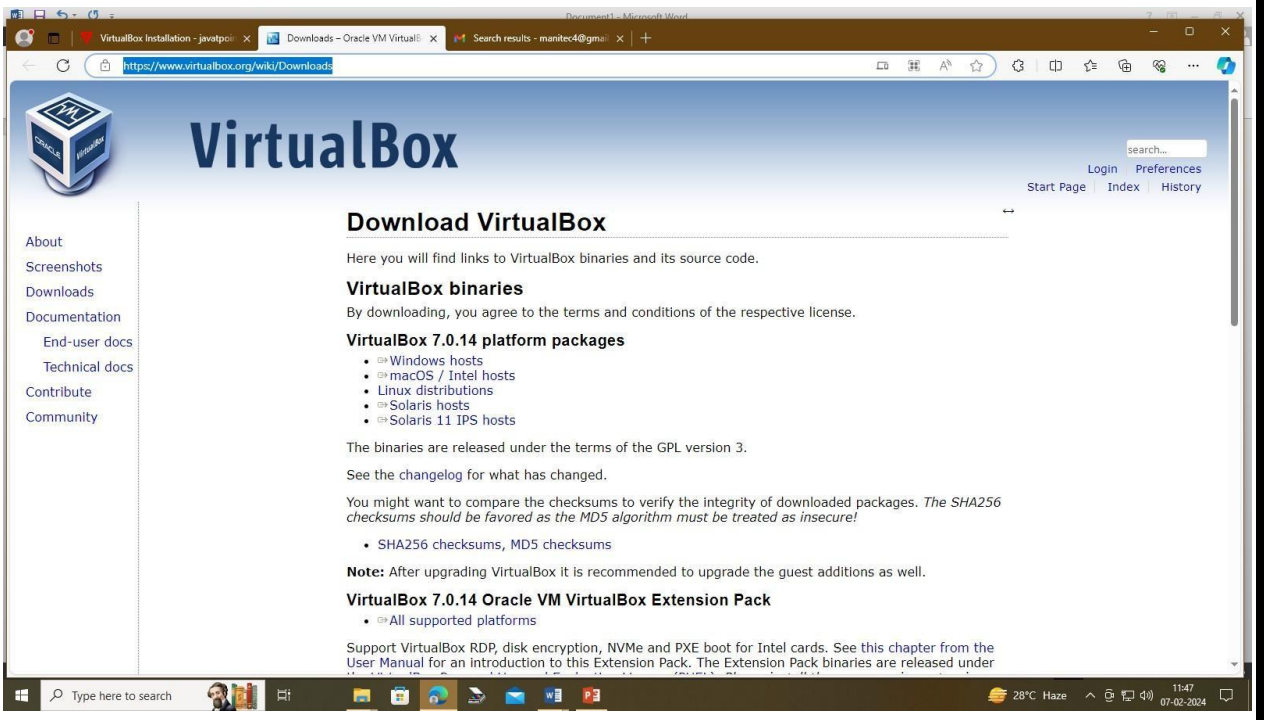
To install Virtualbox/VMware/ Equivalent open source cloud Workstation with different flavours of Linux or Windows OS on top of windows 8 and above.

Hands-on Procedure:

Installation of VirtualBox in Windows

Step 1: First, we need to go to the **VirtualBox Website** through the https://www.virtualbox.org/wiki/Downloads in the system's internet browser. It is the link where we will download the VirtualBox setup file.

Step 2: Click on the **Download VirtualBox** button, which appears on a blue button on the page.



Step 3: Now, Click on the **Windows hosts** link. We will see it below the **"VirtualBox 7.0.14 platform packages"** option.



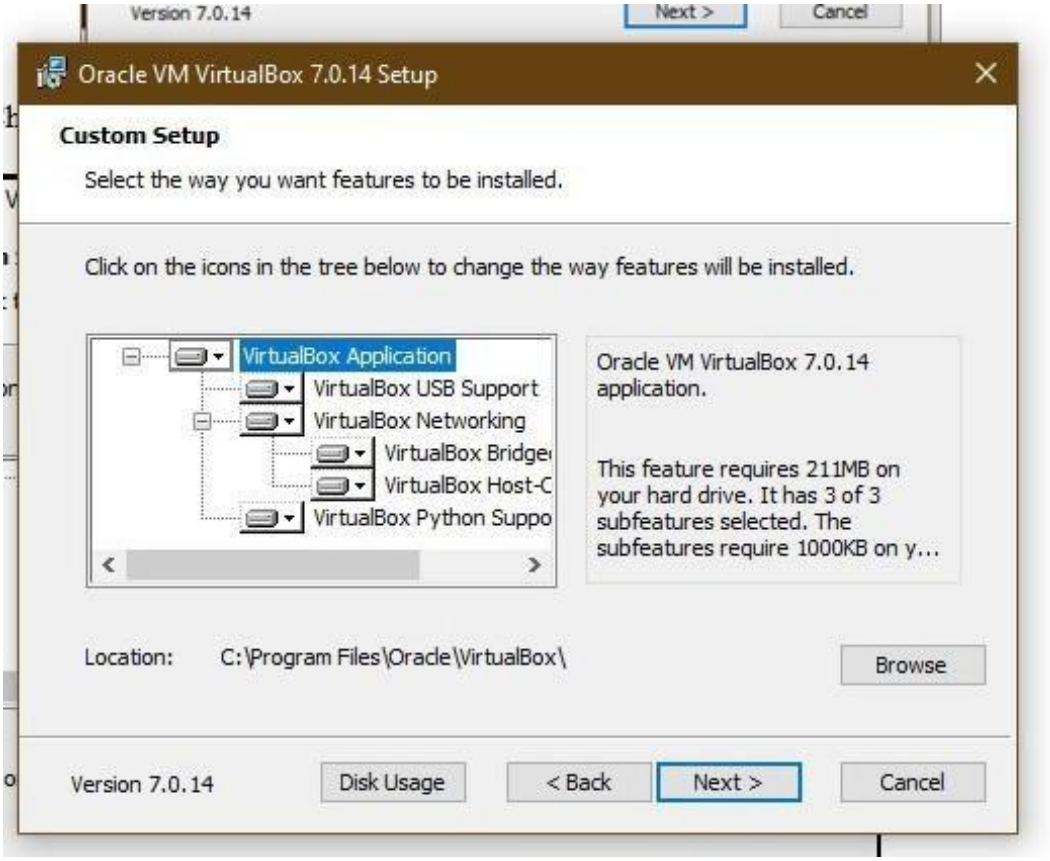
Step 4: Now, open the **VirtualBox EXE** file from where we have downloaded this file in the system. After that, the VirtualBox installation window will open.



Step 5: Click on the **Next** button in the Welcome window.

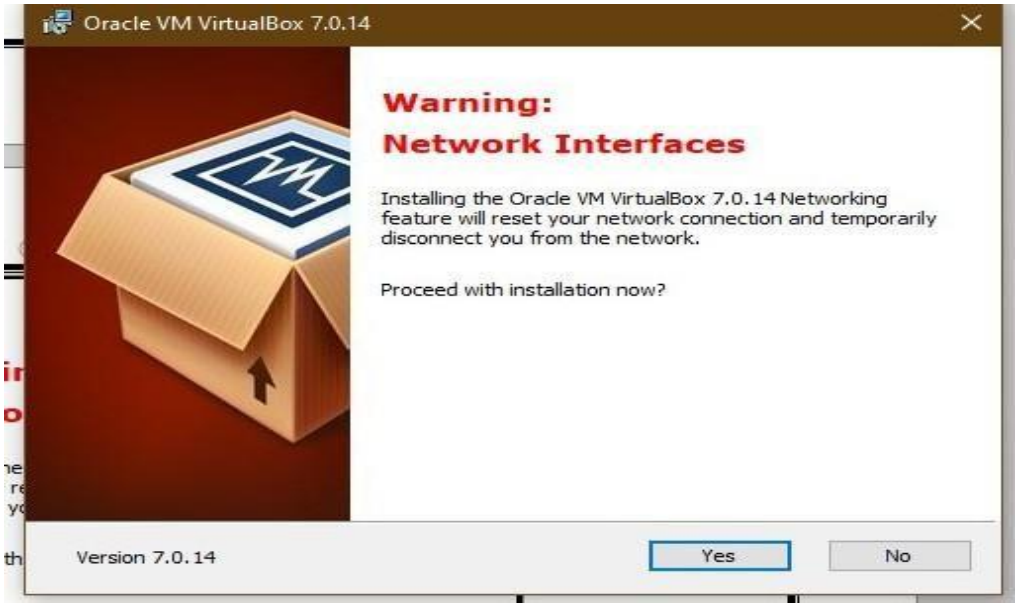


Step 6: Choose the **installation folder** and click on the **Next** button.

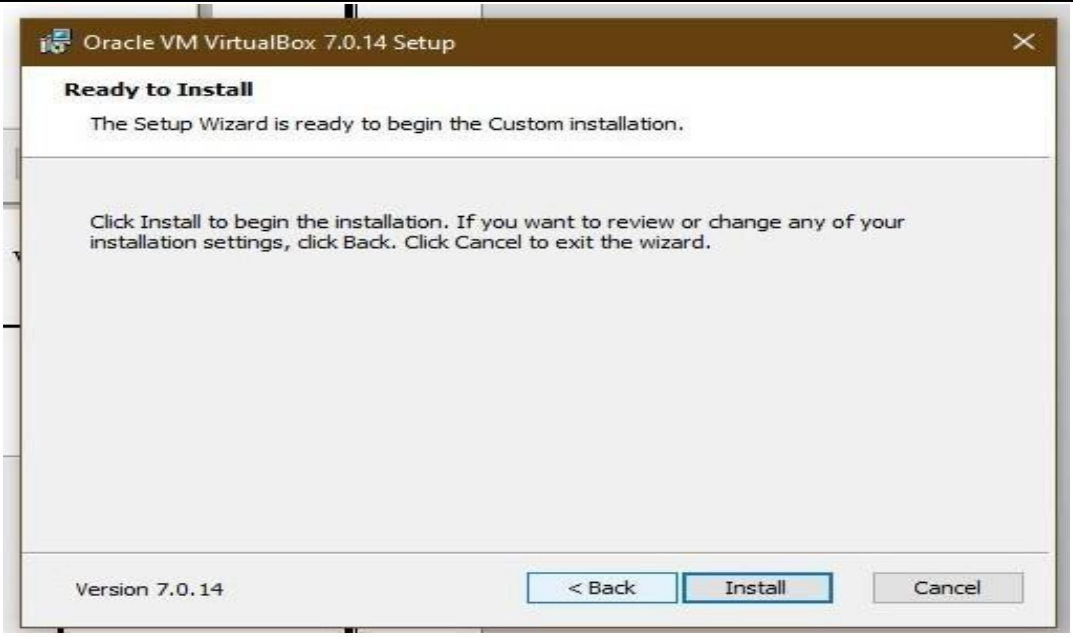


Step 7: Now, we will choose the features that we want to install and then click on the **Next** button.

Step 8: Click on the **Yes** button to install the Oracle VirtualBox interfaces.



Step 9: Click on the **Install** button when it prompted. After that, it will start installing the VirtualBox in our system.



Step 10: After completing all these processes, click on the **Finish** button. When we do this, the installation tab will be closed, and VirtualBox will be opened. Now, we can create a virtual machine to run any OS on the system.



Step11: Open the virtual box and click new Fill the form

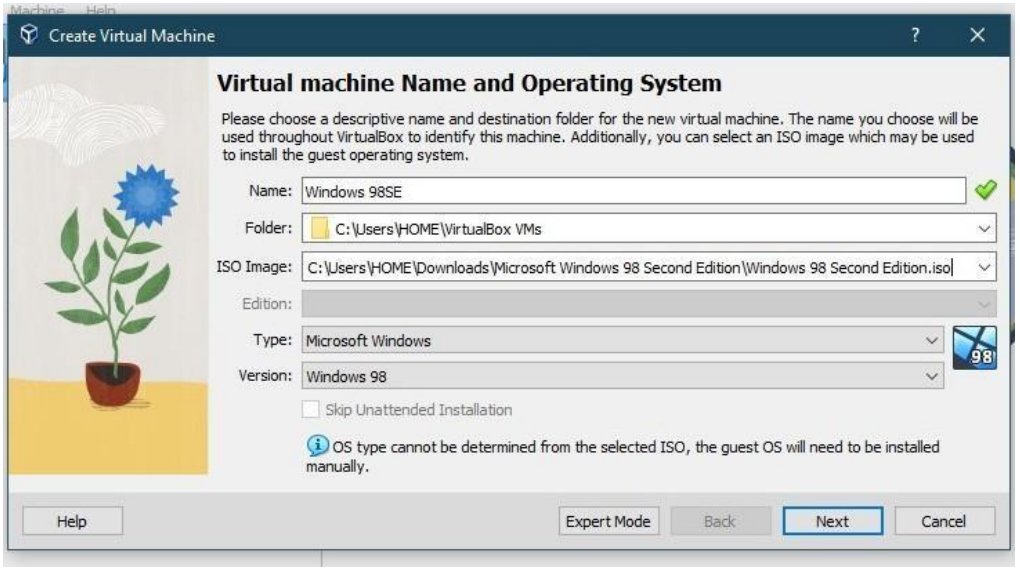


Name: Any name relevant to the Operating system going to install.

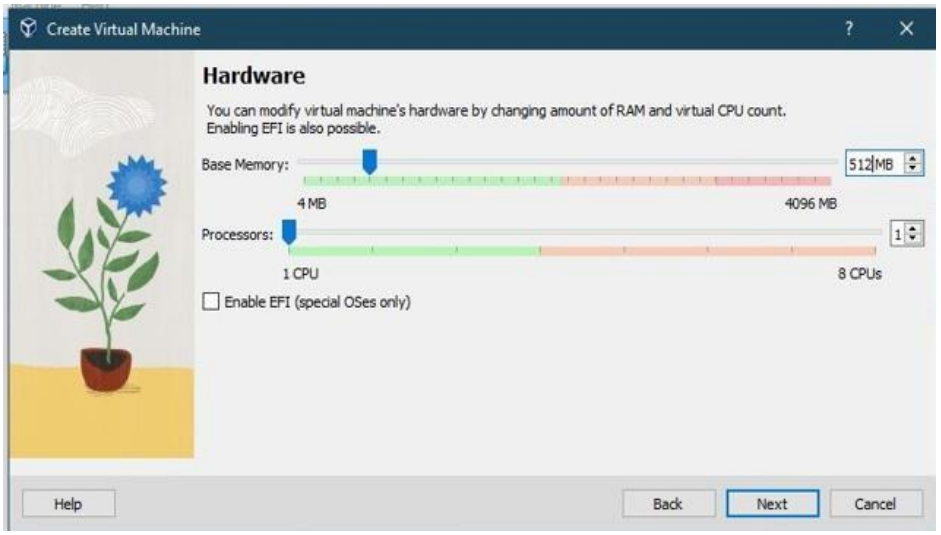
Installation path: Keep default path

ISO Image: Select the ISO image of the Operating System going to install

Type: Select the appropriate OS type from dropdown list (Linux / Windows / Mac).
Version: Select the appropriate OS Version and Architecture of the operating system

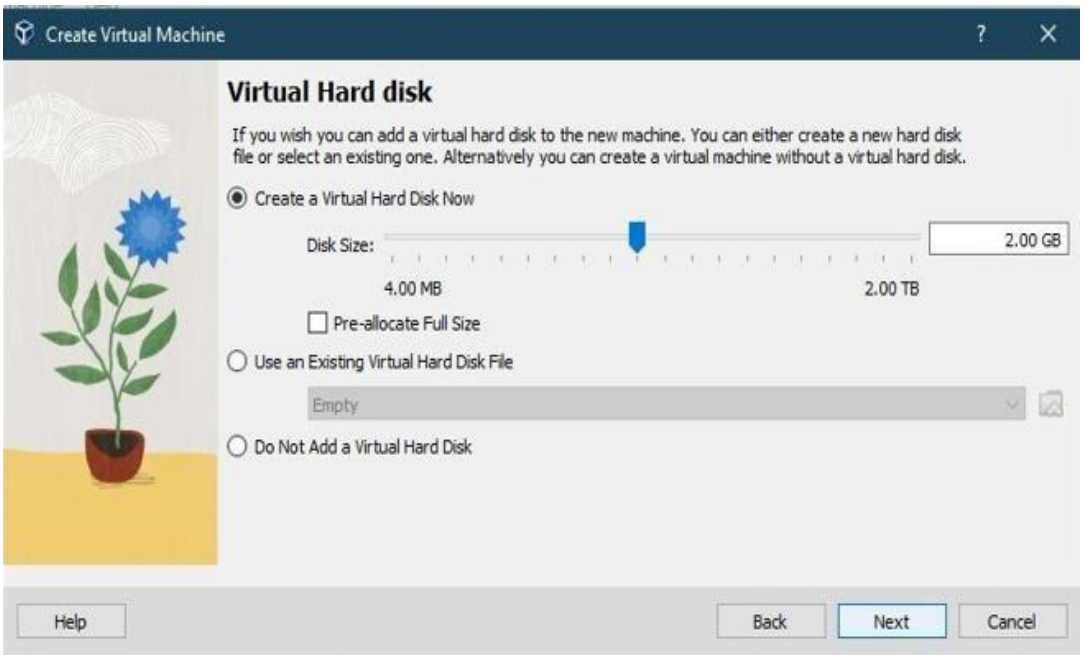


Step 12: Click Next button and allocate the resources to the virtual machine.
Base Memory: 512 MB
Processors: 1 CPU and click the Next Button.



Step13: In this Virtual Hard disk wizard allocate the hard disk space for the virtual machine

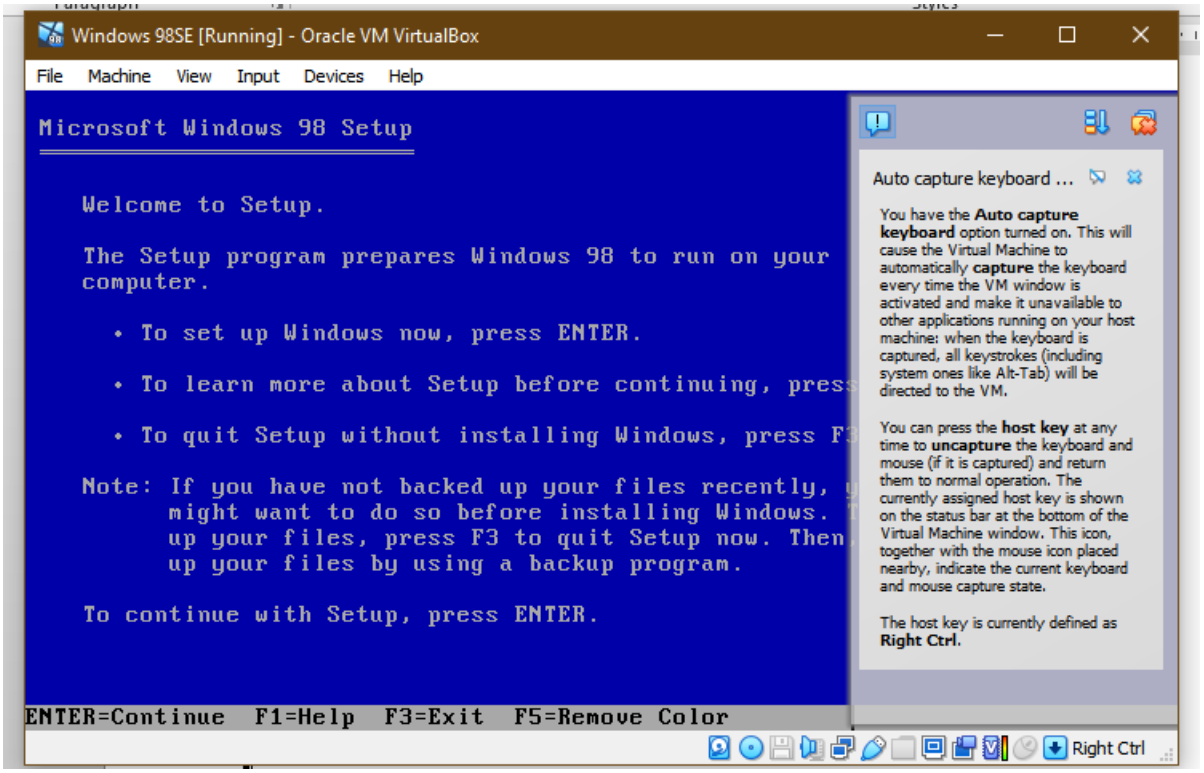
Disk size: 2.00GB and click the Next button, it shows the summary of virtual machine configuration after verifies the configuration click **Finish** Button. It returns to the Virtual Box Manager Home page.

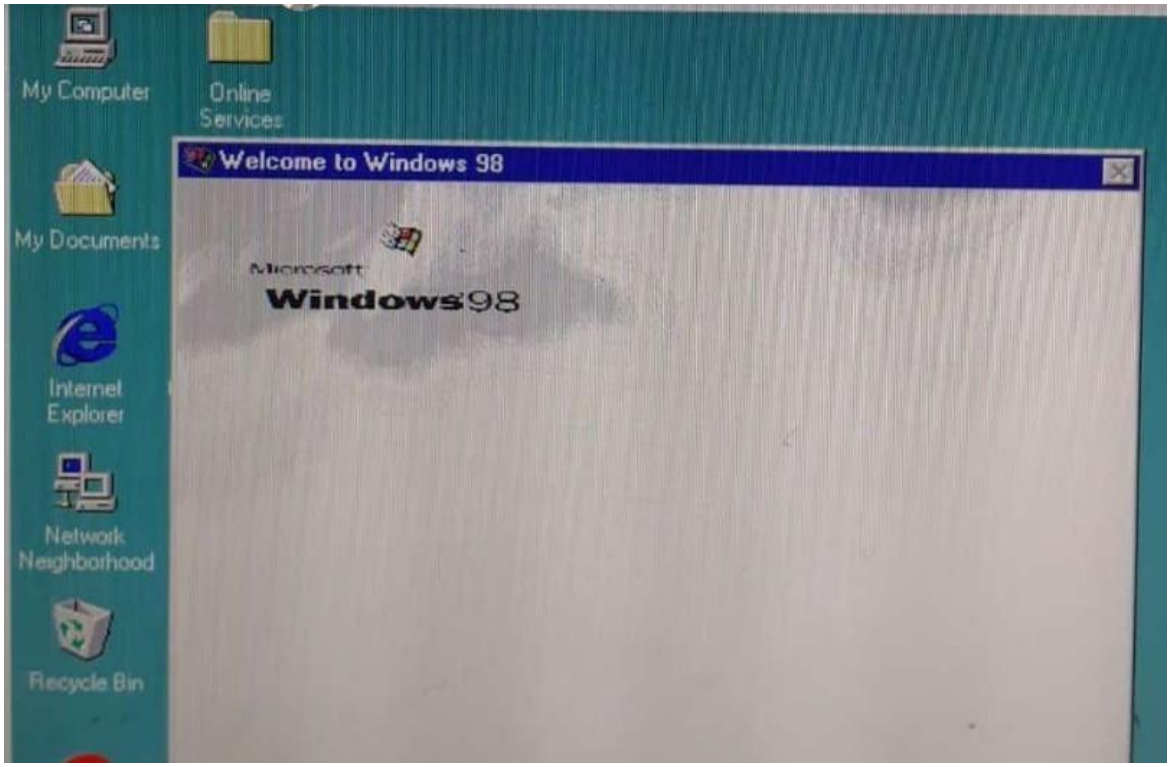


Step14: Select the Virtual operating system and click the Start button to start the windows 98 installation wizard.



Step15: Follow the steps need to install the windows 98, finally it shows the Windows 98 desktop.





Result:

Thus the above VirtualBox installation on Windows 10 Host machine and installation of Windows 98 Guest Operating system are experimented successfully.

Ex.No :02

Virtual Machine – C Compiler

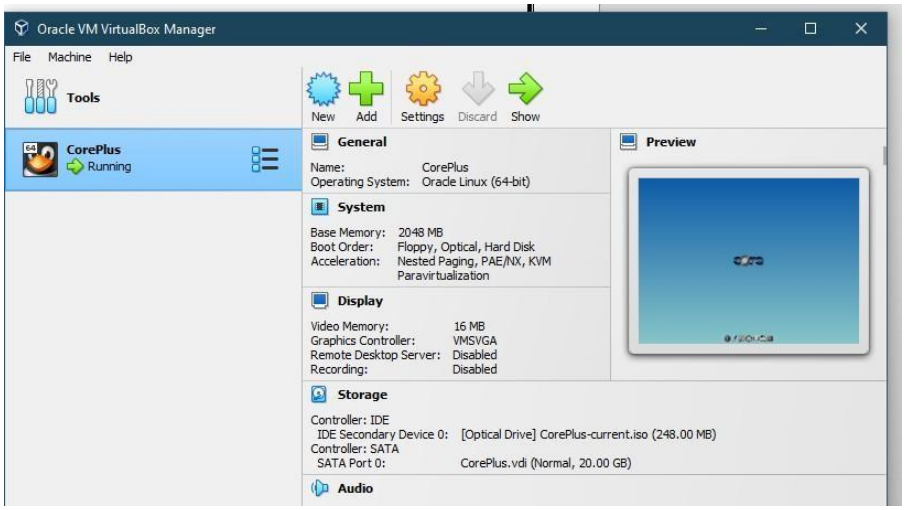
Aim:

Install a C compiler in the virtual machine created using a virtual box and execute Simple Programs.

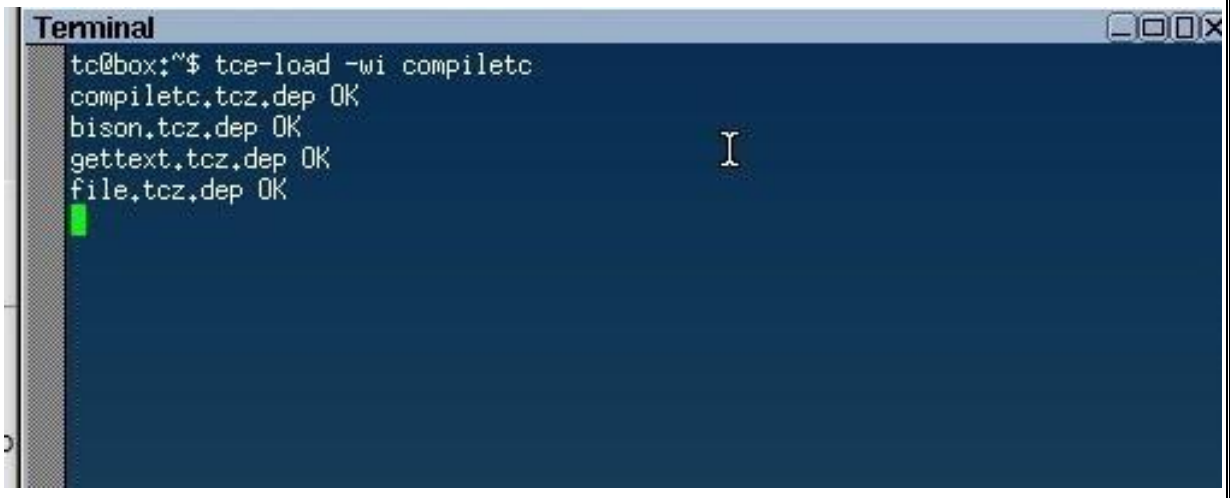
Hands-on Procedure:

STEP 1: Install the VMware player and host the Virtual Machine(CorePlus)

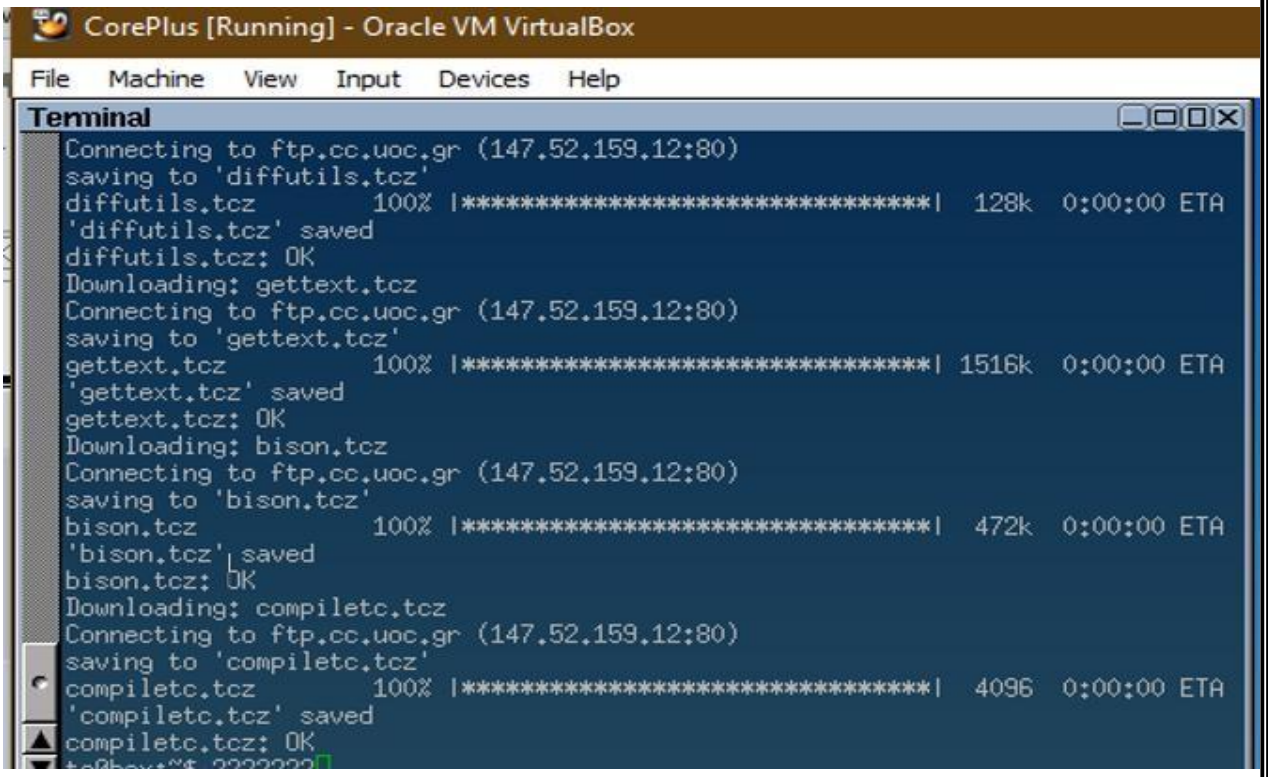
STEP 2: Open the VMware workstation and power on the virtual machine



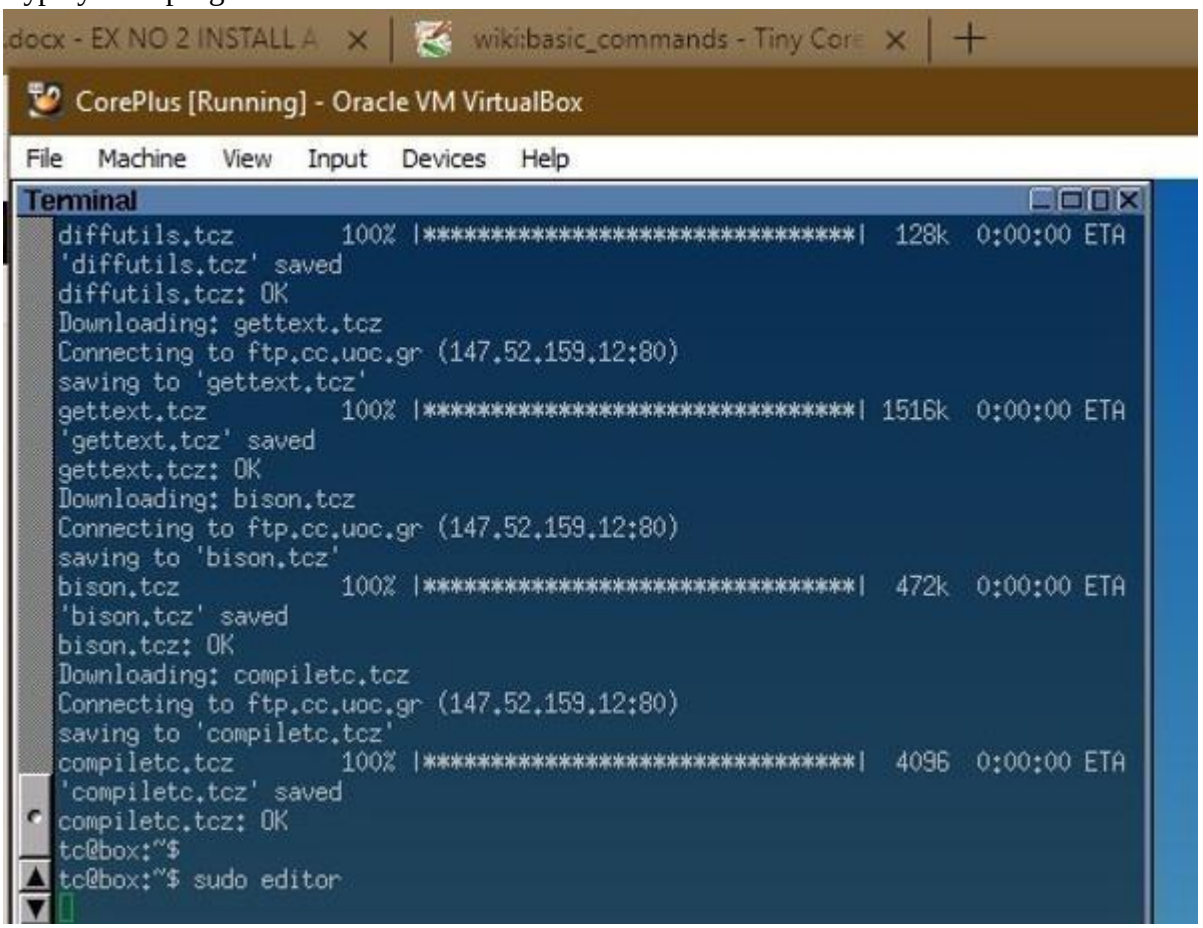
Step2: After loading the Guest Operating Open the terminal and type the command **tc@box:~$ tce-load -wi compiletc**



Step3: After downloading and installing all the library and gcc it return to the prompt.



Step4: Open the Terminal and type the Command **sudo editor** then the text window will appear type your c program and save it with .c extension.



File Edit Search

```
#include<stdio.h>
int main()
{
int y;
printf("Enter the Year");
scanf("%d",&y);
if(y%4==0)
{
if(y%100==0)
{if(y%400==0)
printf("%d is a Leap Year");
else
printf("%d is not Leap Year");
}
else
printf("%d is a Leap Year");
}
else
printf("%d is not Leap Year")
return 0;
}
```

Step5 : After completing the program exit from the editor with CTRL+Q command then the control return to the terminal window.

Step6 : compile the c program with the command cc filename.c command.

tc@box:~$cc demo.c

Step7 : For output use the command ./a.out.

tc@box:~$./a.out

Enter year: 1991

1991 is not a Leap Year

tc@box:~$

Result:

Thus the Install a C compiler in the virtual machine created using a virtual box and execute Simple Programs.

EX NO.:3 Install Google App Engine. Create *hello world* app and other simple web applications using python/java.

Aim:

To Install Google App Engine. Create *hello world* app and other simple web applications using python/java.

Procedure:

1. *Install Google Plugin for Eclipse*

Read this guide – how to install Google Plugin for Eclipse. If you install the Google App Engine Java SDK together with "**Google Plugin for Eclipse**", then go to step 2, Otherwise, get the Google App Engine Java SDK and extract it.

2. *Create New Web Application Project*

In Eclipse toolbar, click on the Google icon, and select "**New Web Application Project…**"

Figure – New Web Application Project

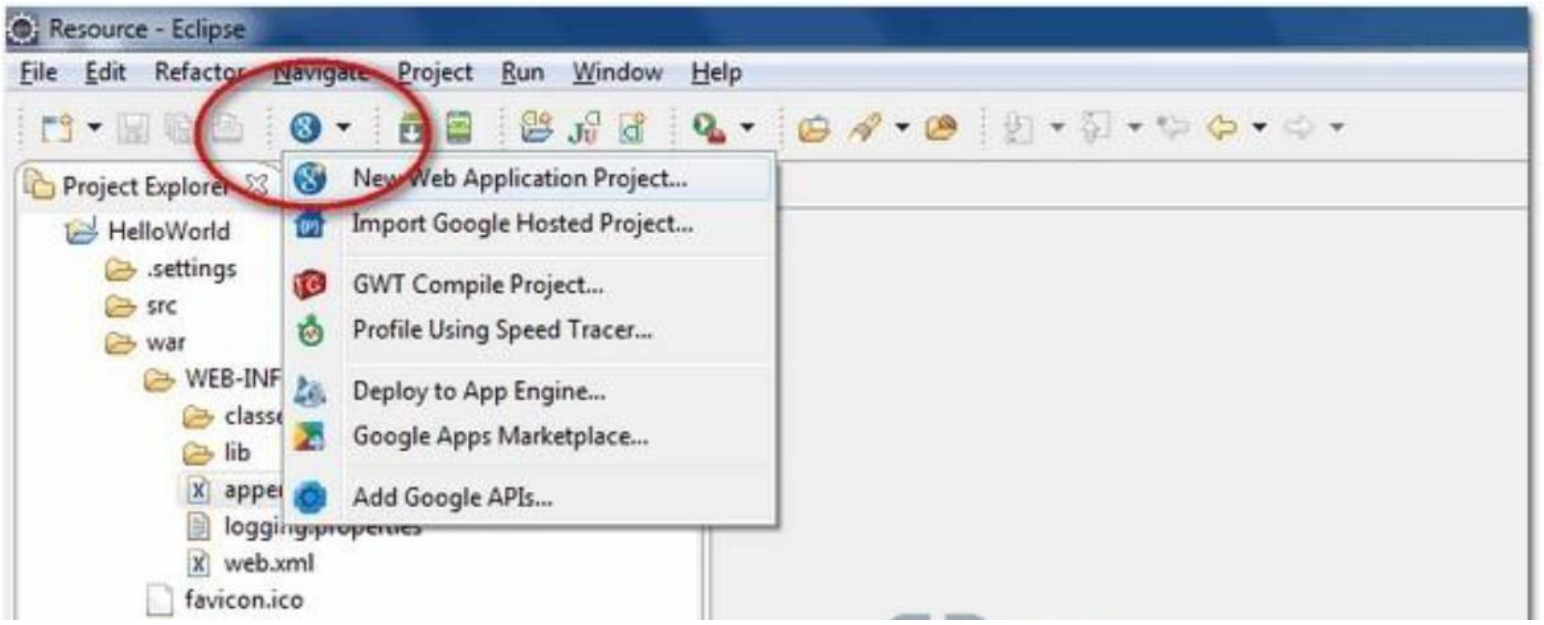
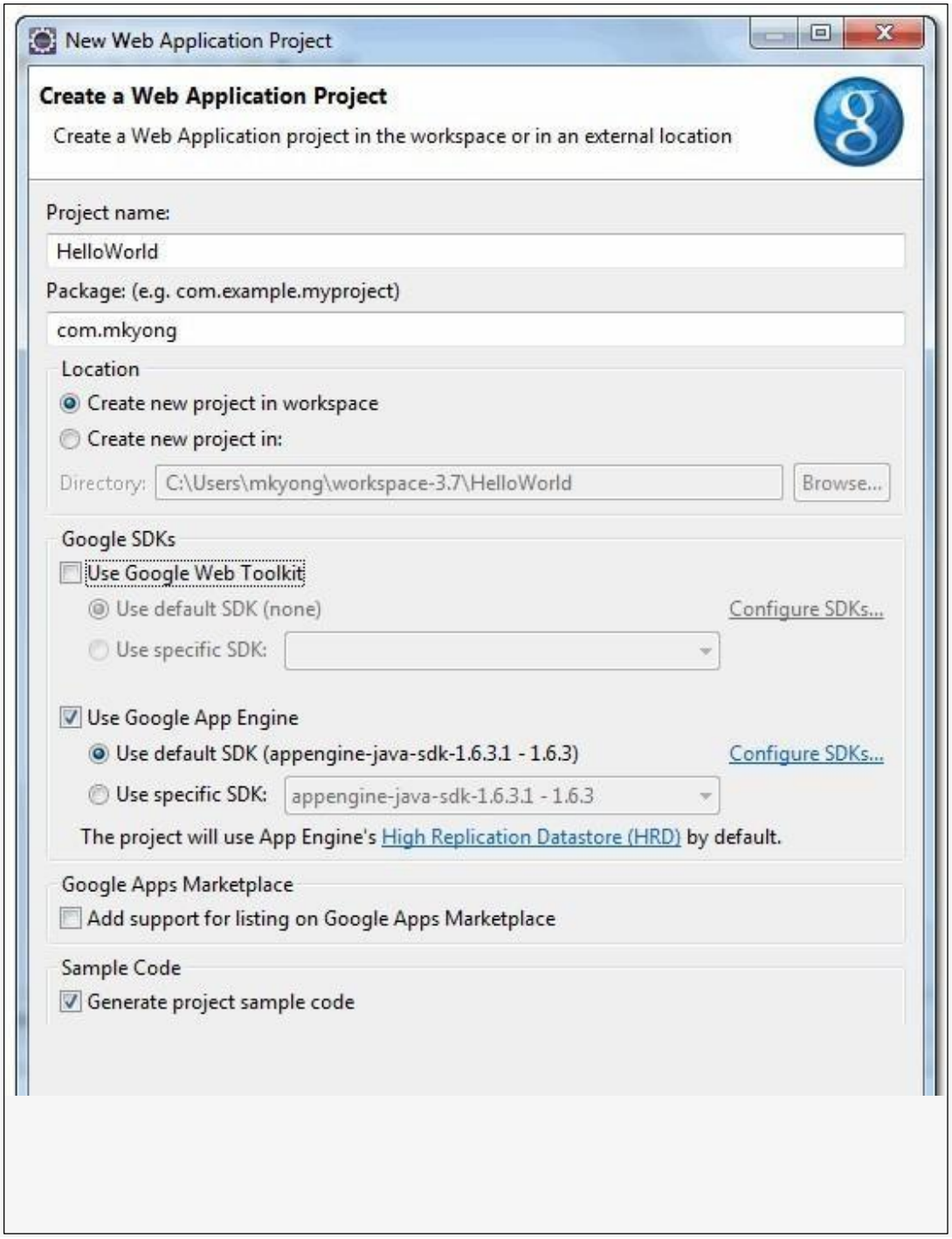


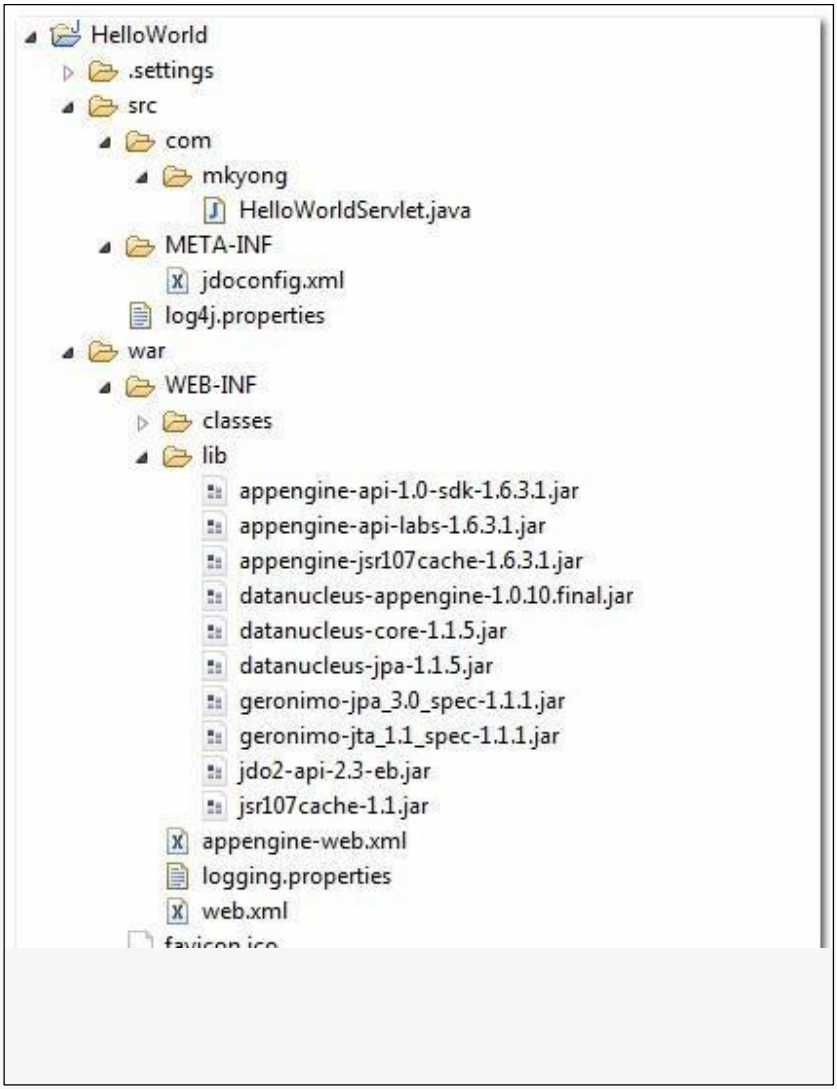
*Figure – Deselect the "**Google Web ToolKit** ", and link your GAE Java SDK via the "**configure SDK**" link.*



Click finished, Google Plugin for Eclipse will generate a sample project automatically.

3. *Hello World*

Review the generated project directory.



Nothing special, a standard Java web project structure.

```
HelloWorld/ src/
    ...Java source code...
    META-INF/
      ...other configuration...
  war/
    ...JSPs, images, data files...
    WEB-INF/
      ...app configuration...
      lib/
        ...JARs for libraries...
      classes/
        ...compiled classes...
```

Copy
The extra is this file "appengine-web.xml", Google App Engine need this to run and deploy the application.

File : appengine-web.xml

```
<?xml version="1.0" encoding="utf-8"?>
<appengine-web-app xmlns="http://appengine.google.com/ns/1.0">
  <application></application>
  <version>1</version>

  <!-- Configure java.util.logging -->
  <system-properties>
    <property name="java.util.logging.config.file" value="WEB-INF/logging.properties"/>
  </system-properties>

</appengine-web-app>
```

Copy

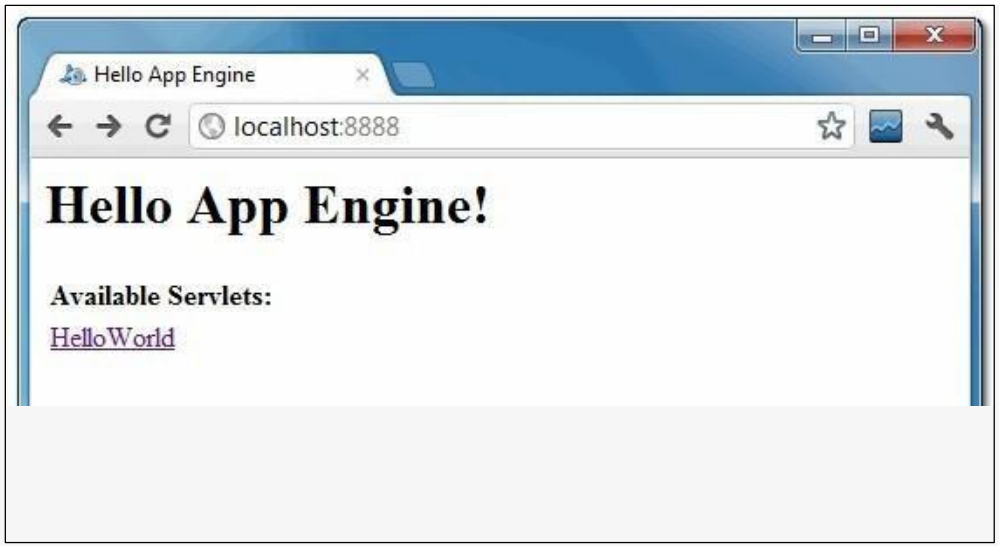
4. *Run it local*

Right click on the project and run as "**Web Application**".

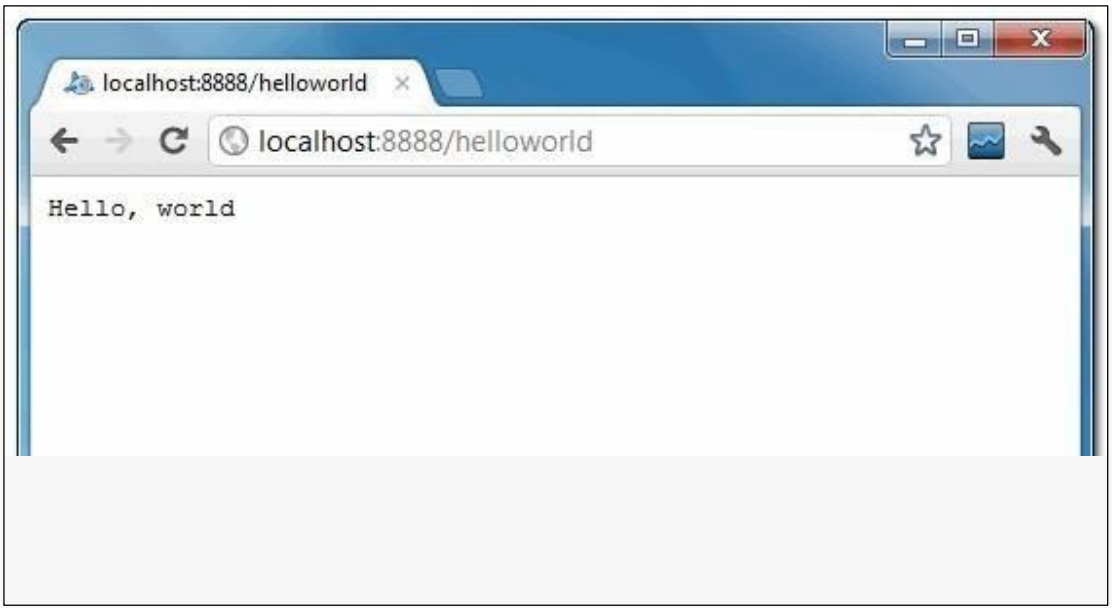
Eclipse console :

```
//...
INFO: The server is running at http://localhost:8888/
30 Mac 2012 11:13:01 PM com.google.appengine.tools.development.DevAppServerImpl start INFO: The admin console is running at http://localhost:8888/_ah/admin
```

Copy
Access URL http://localhost:8888/, see output



and also the hello world servlet – http://localhost:8888/helloworld



5. *Deploy to Google App Engine*

Register an account on https://appengine.google.com/, and create an application ID for your web application.

In this demonstration, I created an application ID, named "mkyong123", and put it in appengine- web.xml.

File : appengine-web.xml

```
<?xml version="1.0" encoding="utf-8"?>
<appengine-web-app xmlns="http://appengine.google.com/ns/1.0">
  <application>mkyong123</application>
  <version>1</version>

  <!-- Configure java.util.logging -->
  <system-properties>
    <property name="java.util.logging.config.file" value="WEB-INF/logging.properties"/>
  </system-properties>

</appengine-web-app>
```

Copy

To deploy, see following steps:

Figure 1.1 – Click on GAE deploy button on the toolbar.

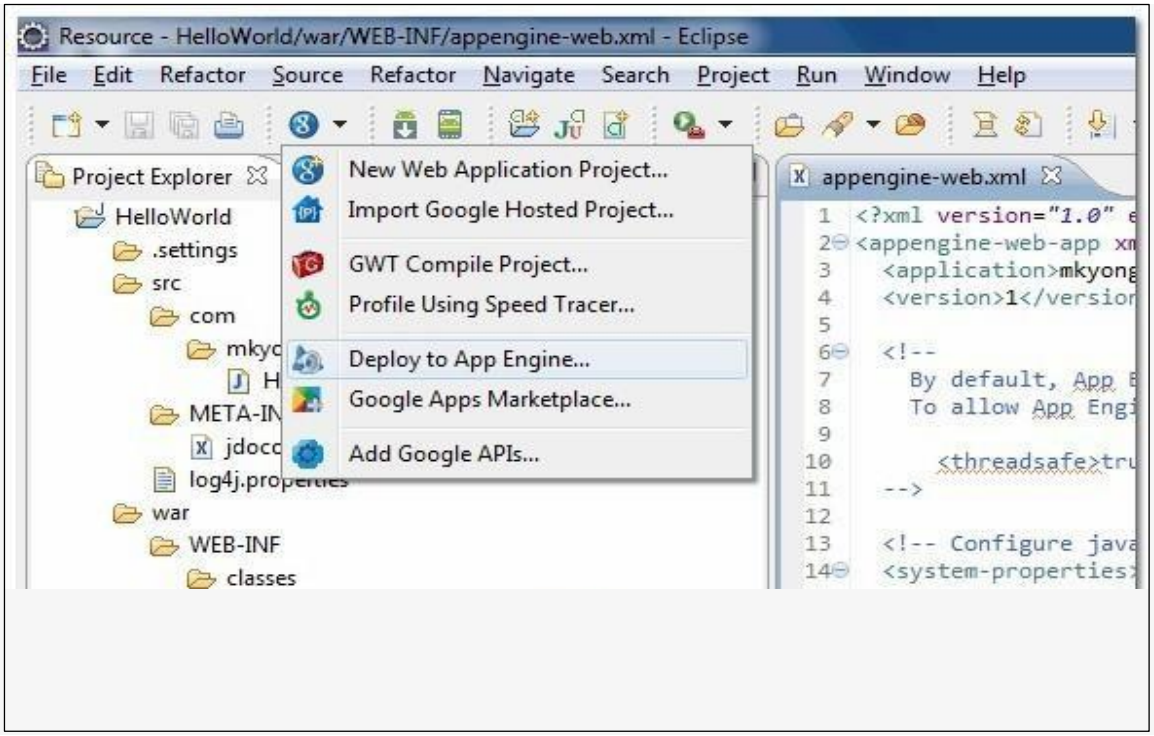


Figure 1.2 – Sign in with your Google account and click on the Deploy button.

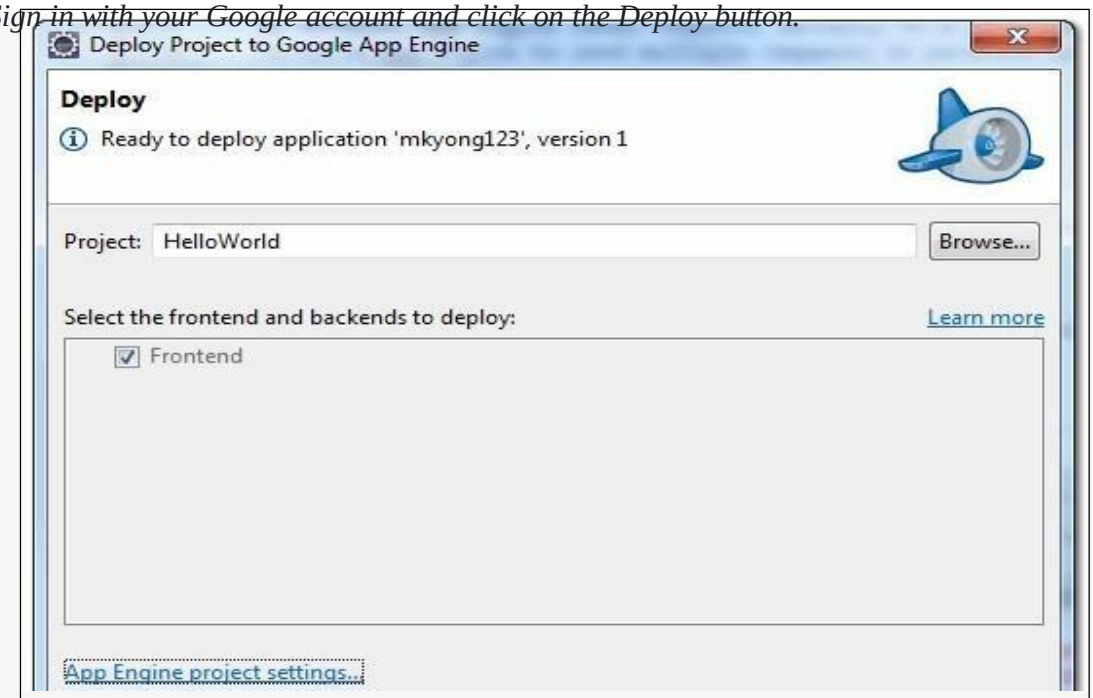


Figure 1.3 – If everything is fine, the hello world web application will be deployed to this URL – http://mkyong123.appspot.com/



Result:

Thus the Google App Engine is installed successfully and a web application to display hello world using python is developed and deployed in the GAE .

Ex.No:4 Date:	Use GAE launcher to launch the web applications

AIM:

To Use GAE launcher to launch the web applications

PROCEDURE:

STEP:1 Create a new Cloud Console project or retrieve the project ID of an existing project to use

STEP:2 Go to the project page

STEP:3 Install and then initialize the Google Cloud SDK

STEP:4 Download SDK

STEP:5 Creating a website to host on Google App Engine

STEP:6 Basic structure for the project

STEP:7 This guide uses the following structure for the project:

- app.yaml: Configure the settings of your App Engine application.
- www/: Directory to store all of your static files, such as HTML, CSS, images, and JavaScript.
- css/: Directory to store stylesheets.
- style.css: Basic stylesheet that formats the look and feel of your site.
- images/: Optional directory to store images.
- index.html: An HTML file that displays content for your website.
- js/: Optional directory to store JavaScript files.
- Other asset directories.

STEP: 8 Creating the app.yaml file

- ✓ The app.yaml file is a configuration file that tells App Engine how to map URLs to your static files. In the following steps, you will add handlers that will load www/index.html when someone visits your website, and all static files will be stored in and called from the www directory.
- ✓ Create the app.yaml file in your application's root directory:

- ✓ Create a directory that has the same name as your project ID. You can find your project ID in the Console.
- ✓ In directory that you just created, create a file named app.yaml.
- ✓ Edit the app.yaml file and add the following code to the file:

```
runtime: python27
api_version: 1
threadsafe: true

handlers:
- url: /
  static_files: www/index.html
  upload: www/index.html

- url: /(.*)
  static_files: www/\1
  upload: www/(.*)
```

More reference information about the app.yaml file can be found in the app.yaml reference documentation.

STEP: 9 creating the index.html file

- ✓ Create an HTML file that will be served when someone navigates to the root page of your website. Store this file in your www directory.

```
<html>
  <head>
    <title>Hello, world!</title>
    <link rel="stylesheet" type="text/css" href="/css/style.css">
  </head>
  <body>
    <h1>Hello, world!</h1>
    <p>
      This is a simple static HTML file that will be served from Google App
      Engine.
    </p>
  </body>
</html>
```

STEP: 10 Deploying your application to App Engine

- ✓ When you deploy your application files, your website will be uploaded to App Engine. To deploy your app, run the following command from within the root directory of your application where the app.yaml file is located:
- ✓ gcloud app deploy
- ✓ Optional flags:
- ✓ Include the --project flag to specify an alternate Cloud Console project ID to what you initialized as the default in the gcloud tool. Example: --project [YOUR_PROJECT_ID]

- ✓ Include the -v flag to specify a version ID, otherwise one is generated for you. Example: -v [YOUR_VERSION_ID]
- ✓ To learn more about deploying your app from the command line, see Deploying a Python 2 App.

STEP: 11 Viewing your application

- ✓ To launch your browser and view the app at https://PROJECT_ID.REGION_ID.r.appspot.com, run the following command:

gcloud app browse

RESULT:

Thus a GAE launcher is used to launch the web applications and successfully executed.

Ex.No:5 Date:	Simulate a cloud scenario using CloudSim

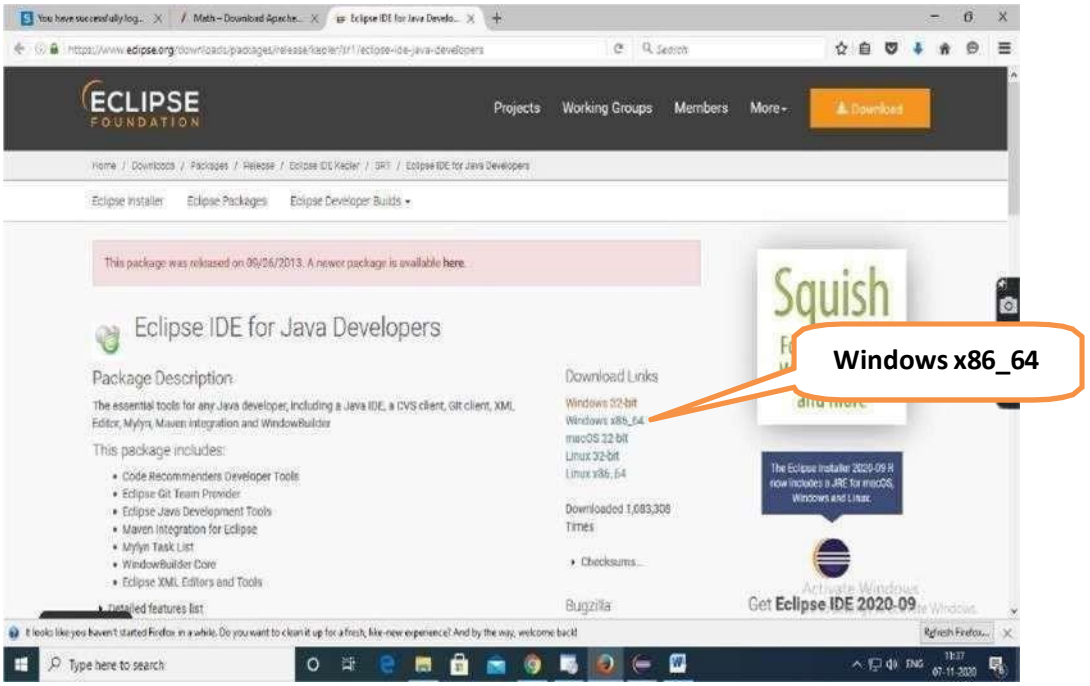
AIM:

To Simulate a cloud scenario using CloudSim and run a scheduling algorithm that is not present in CloudSim

PROCEDURE:

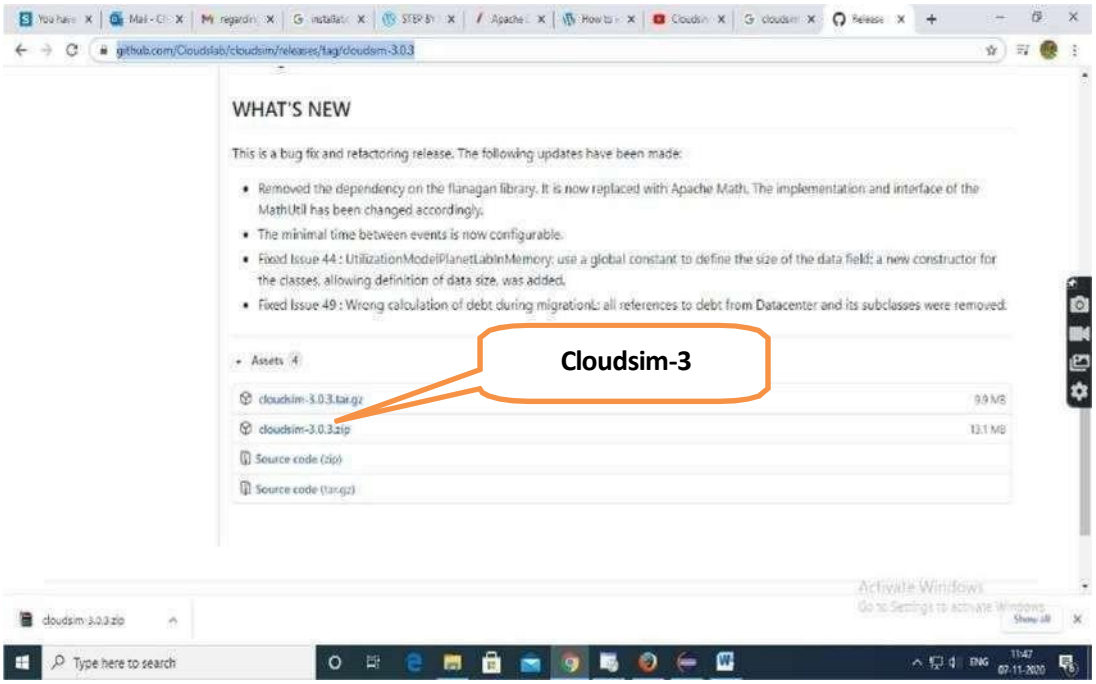
Step 1: Link to download Eclipse and download Eclipse for Windows 64bit into your Local machine

https://www.eclipse.org/downloads/packages/release/kepler/sr1/eclipse-ide-java-developers



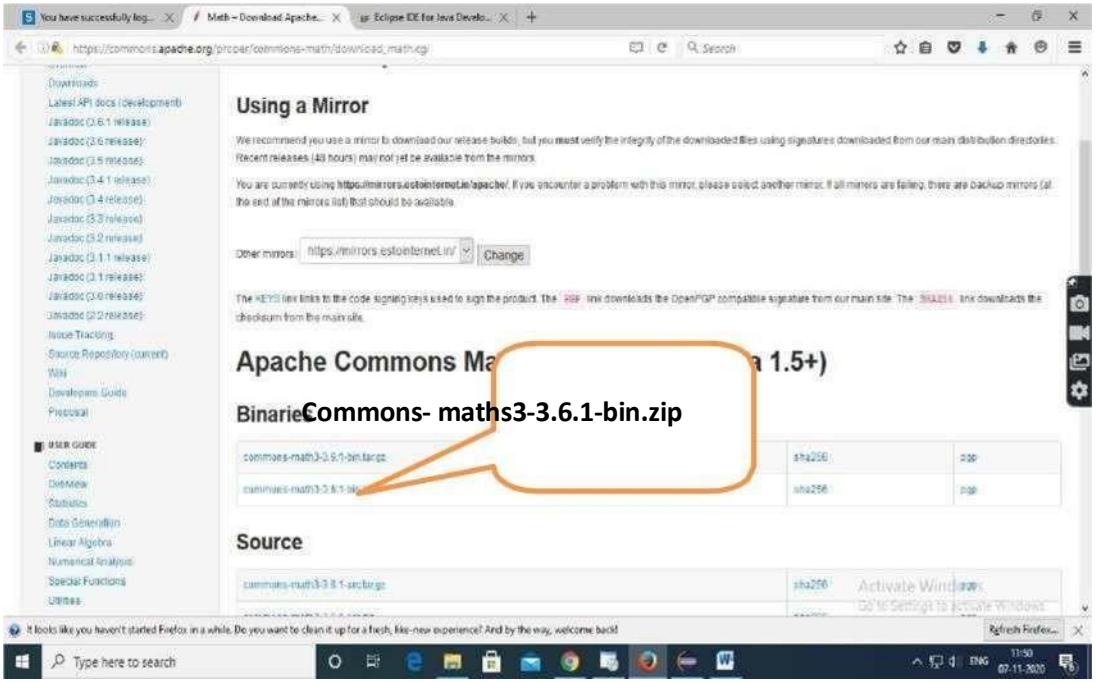
Step 2: Download cloudsim-3.0.3 from git hub repository in your local machine

https://github.com/Cloudslab/cloudsim/releases/tag/cloudsim-3.0.3

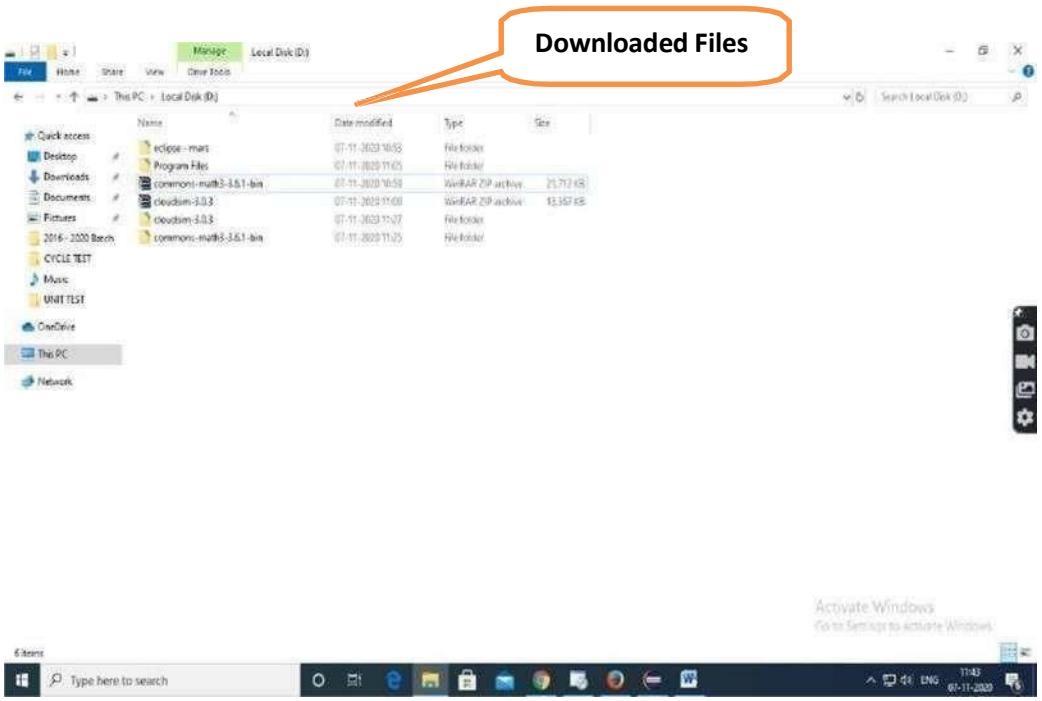


Step 3: Download commons-maths3-3.6.1 from git hub repository in your local machine

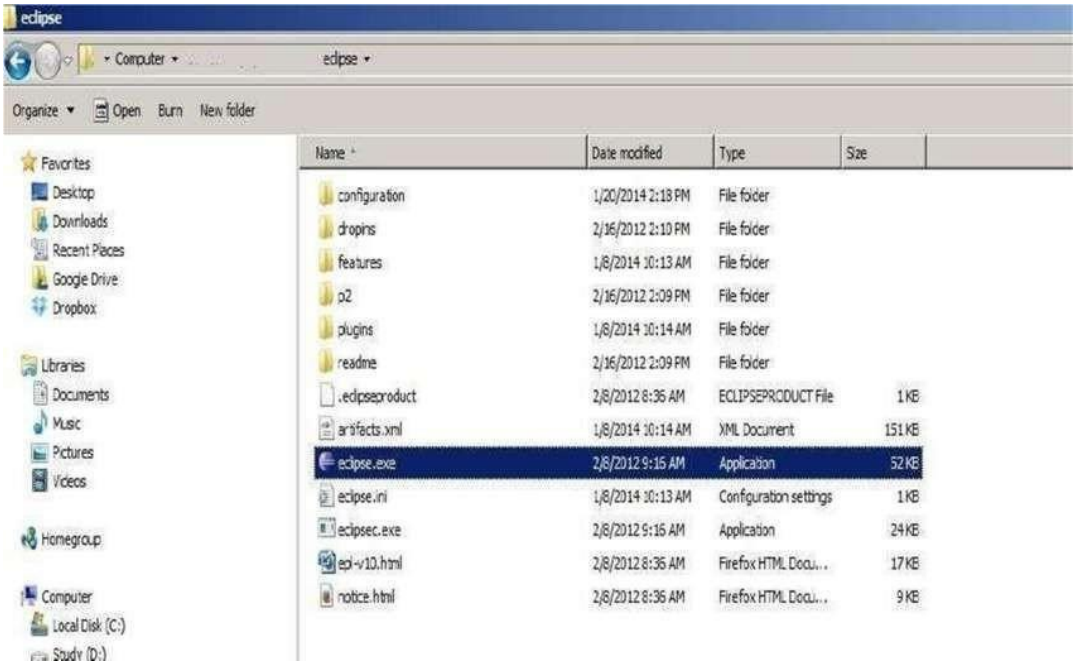
https://commons.apache.org/proper/commons-math/download_math.cgi



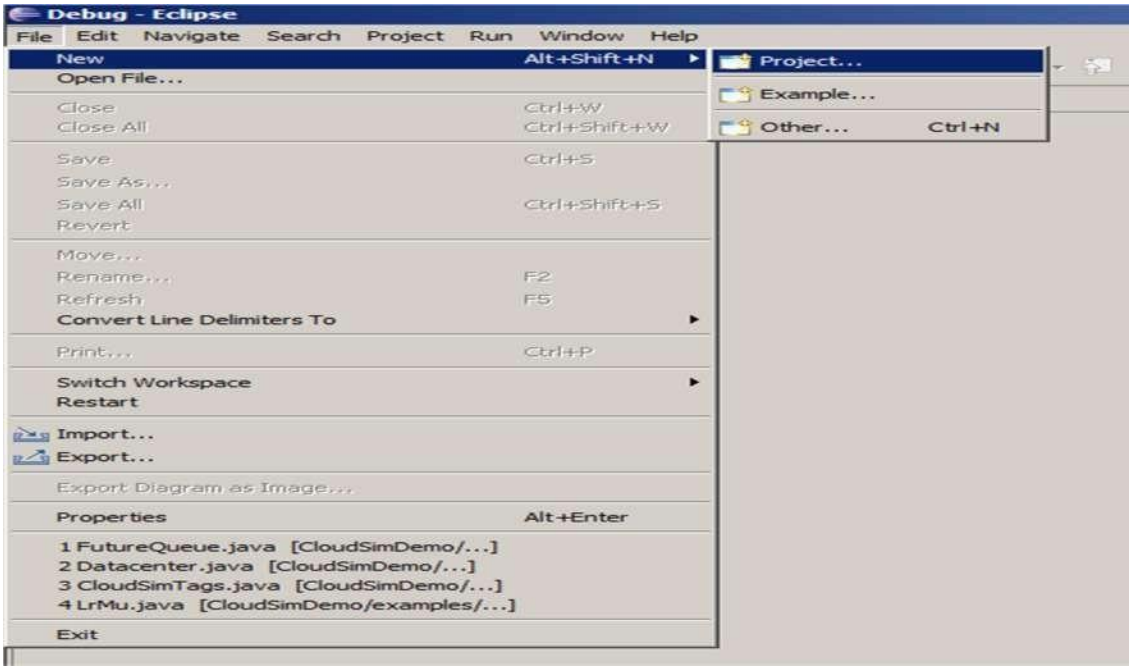
Step 4: Downloaded Eclipse, cloudsim-code-master and Apache Commons Math 3.6.1 in your local machine and extract cloudsim-3.0.3 and Apache Commons Math 3.6.1



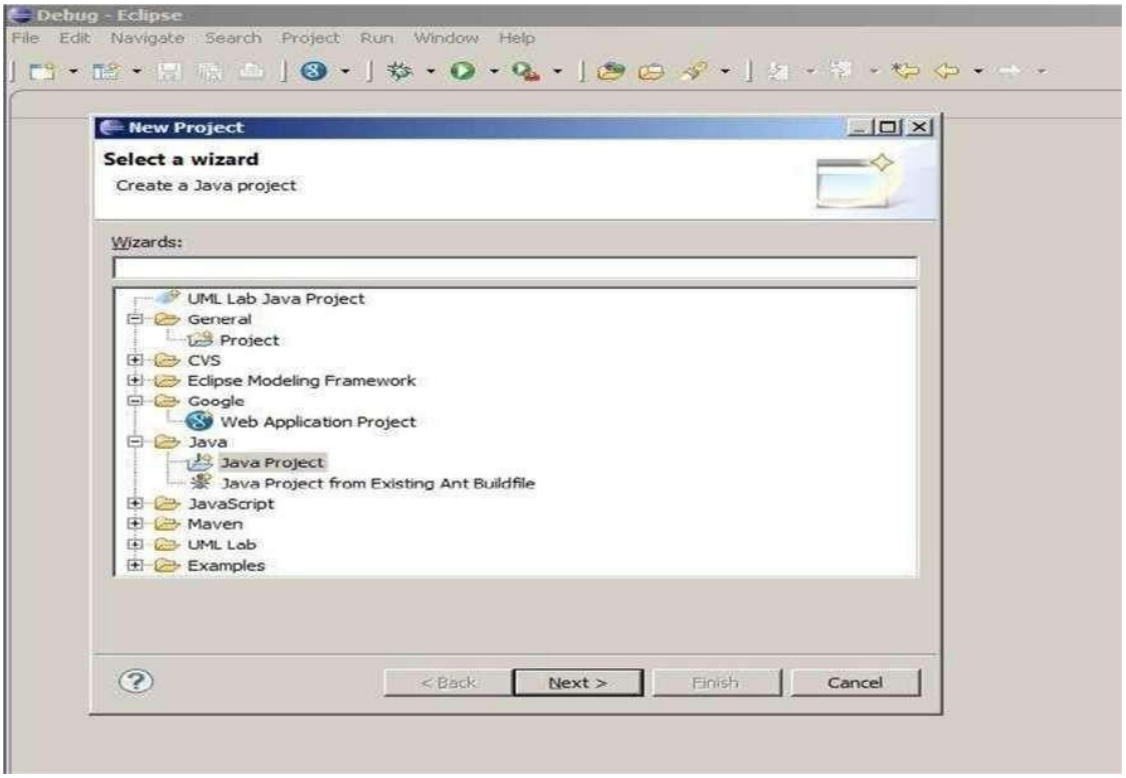
Step 5: First of all, navigate to the folder where you have unzipped the eclipse folder and open Eclipse.exe



Step 6: Now within Eclipse window navigate the menu: File -> New -> Project, to open the new project wizard

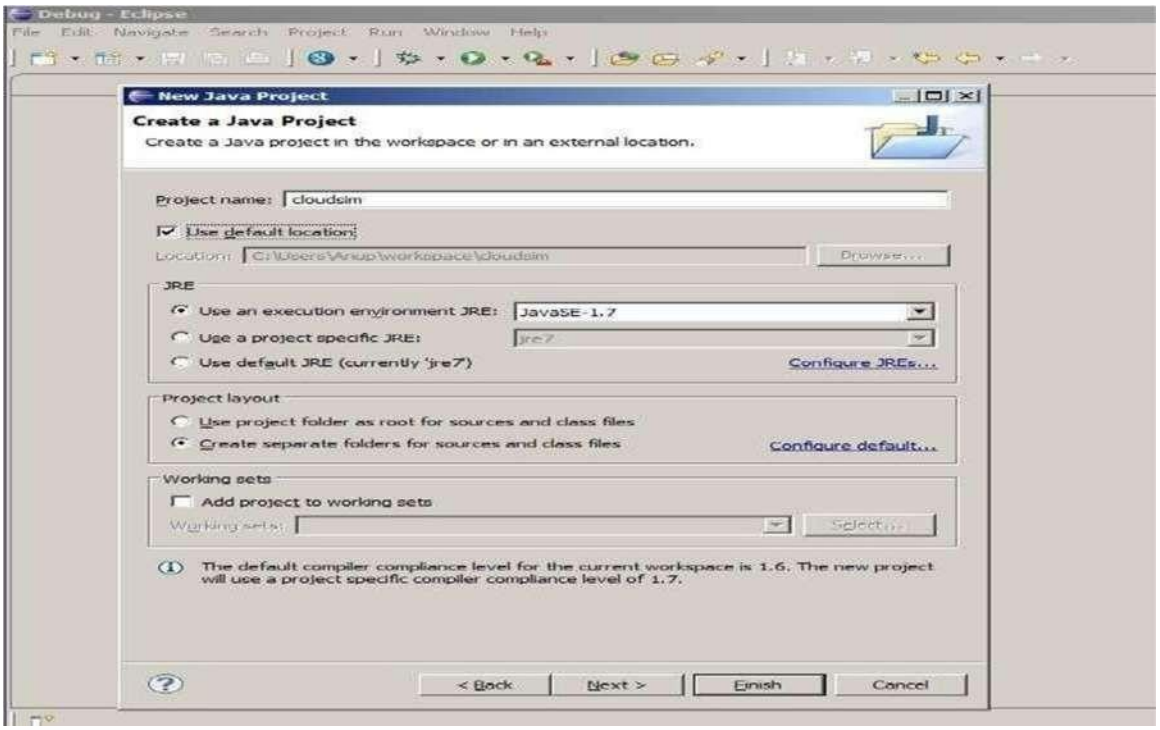


Step 7: A _New Project_ wizard should open. There are a number of options displayed and you have to find & select the _Java Project_ option, once done click 'Next_

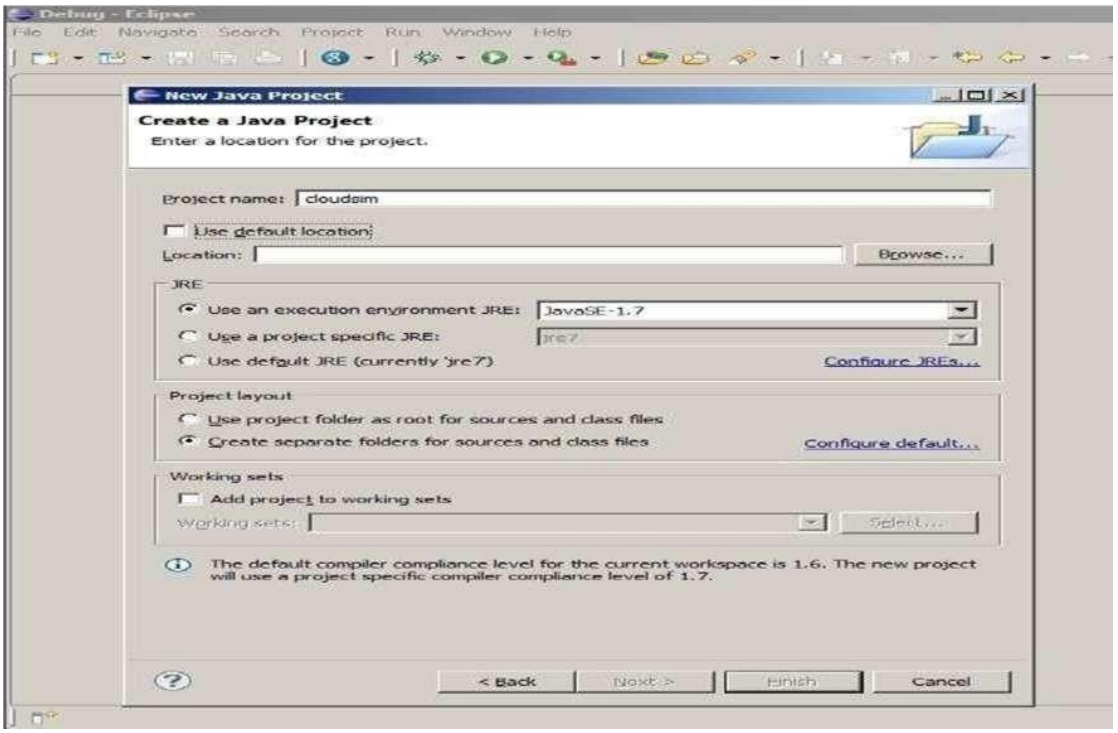


Step 8: Now a detailed new project window will open, here you will provide the project name and the path of CloudSim project source code, which will be done as follows:

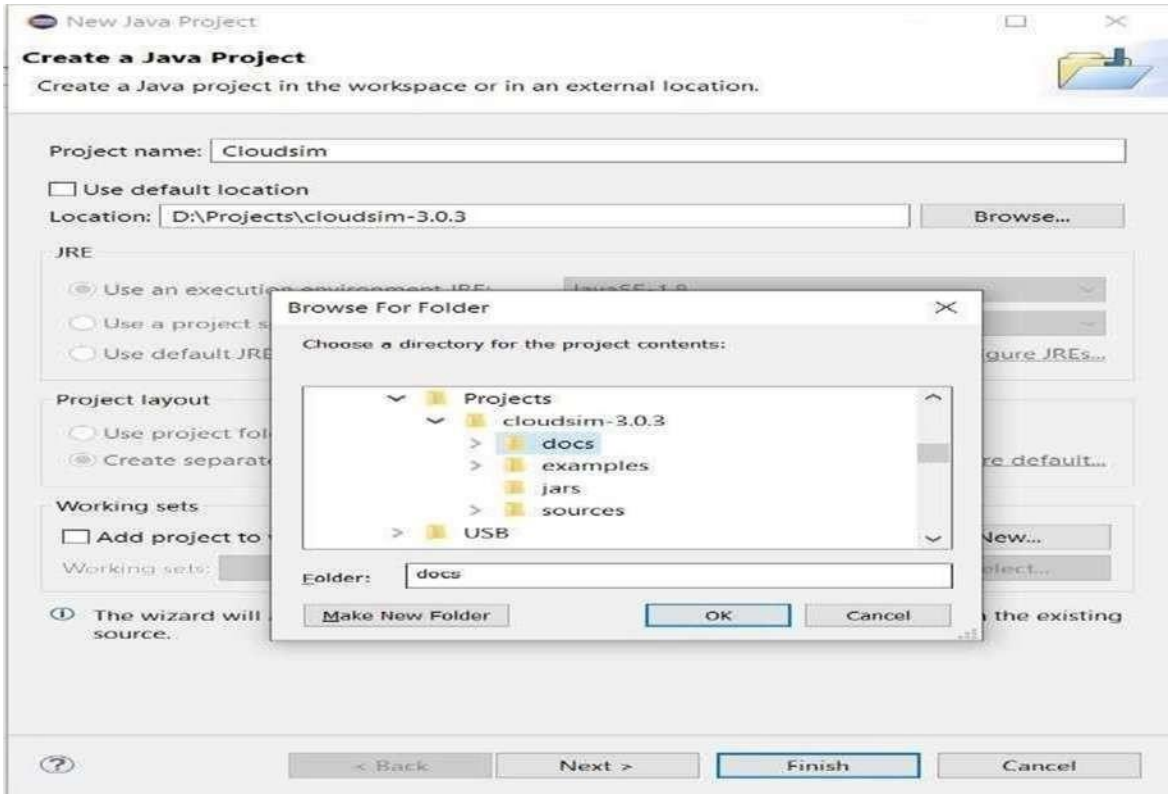
Project Name: CloudSim.



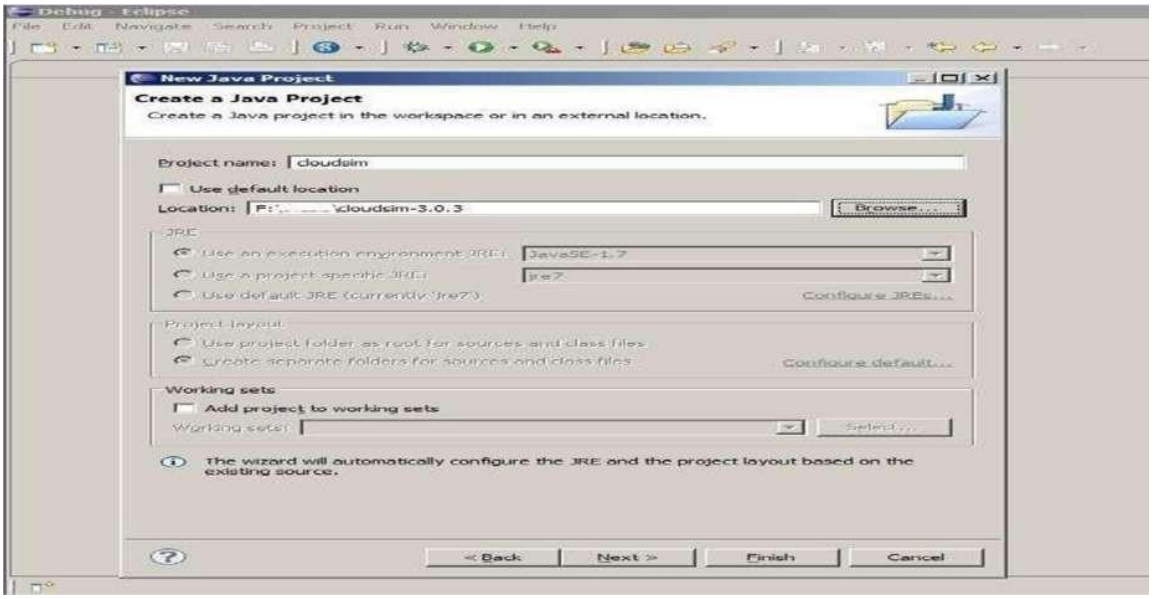
Step 9: Unselect the 'Use default location' option and then click on 'Browse' to open the path where you have unzipped the Cloudsim project and finally click Next to set project settings.



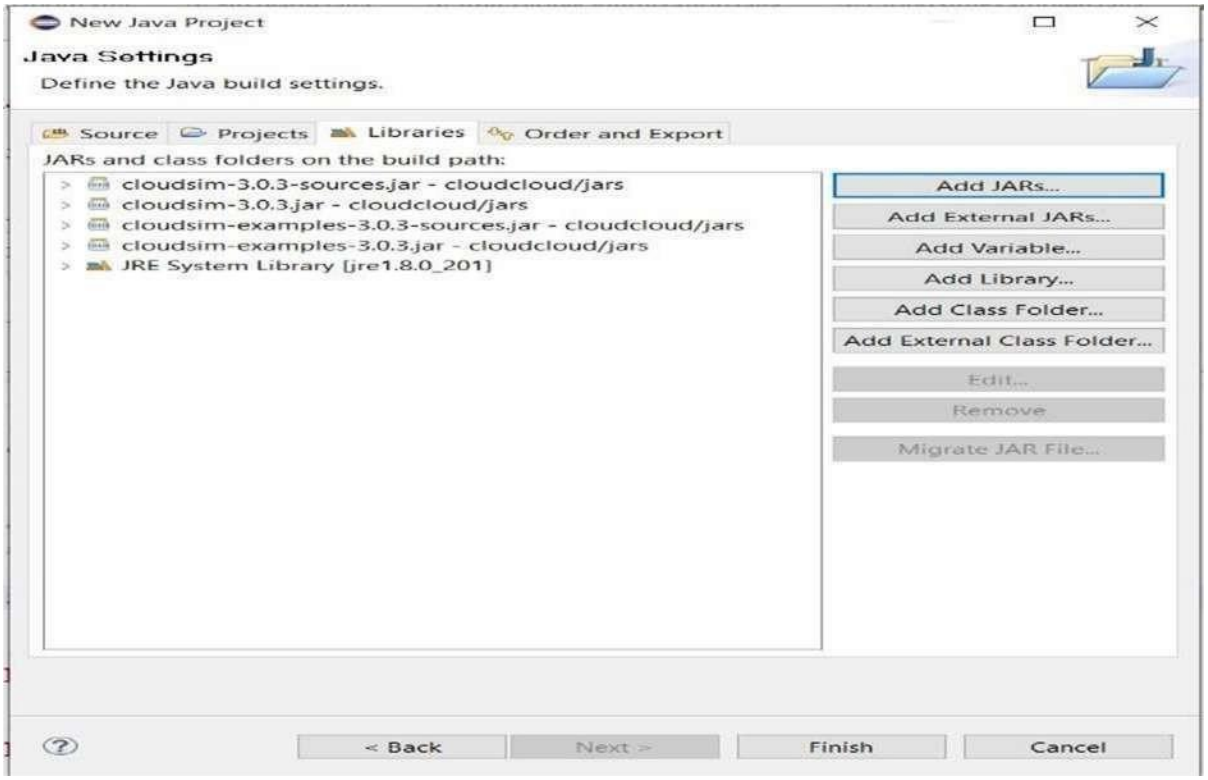
Step 10: Make sure you navigate the path till you can see the bin, docs, examples etc folder inthe navigation plane.



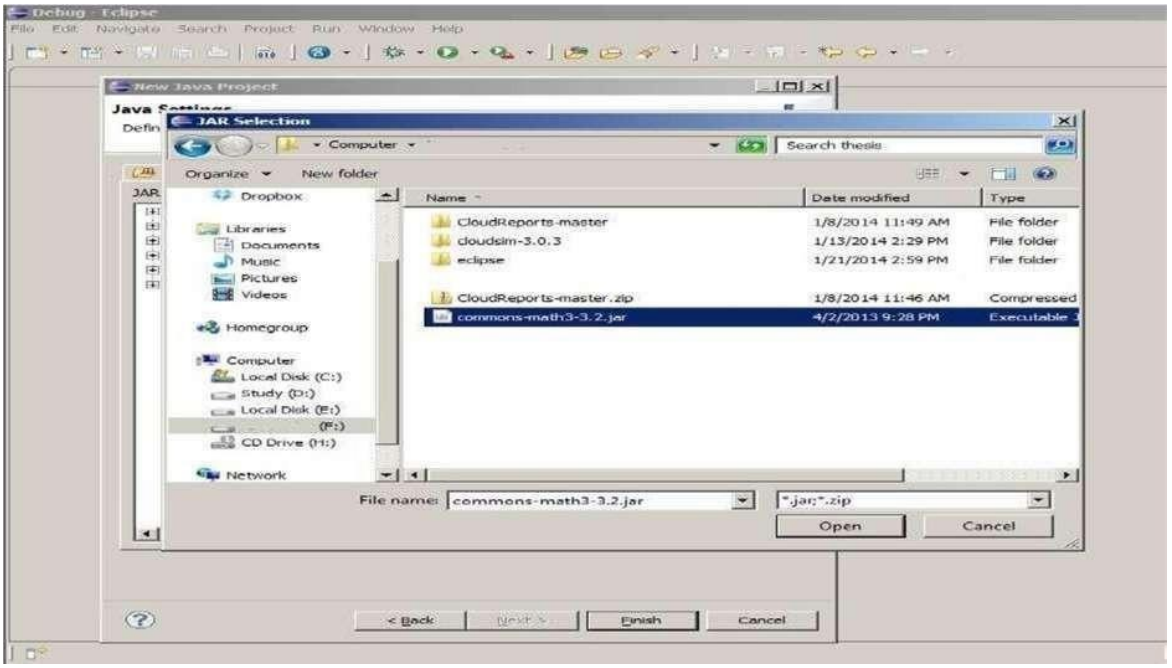
Step 11: Once done finally, click 'Next' to go to the next step i.e. setting up of project settings



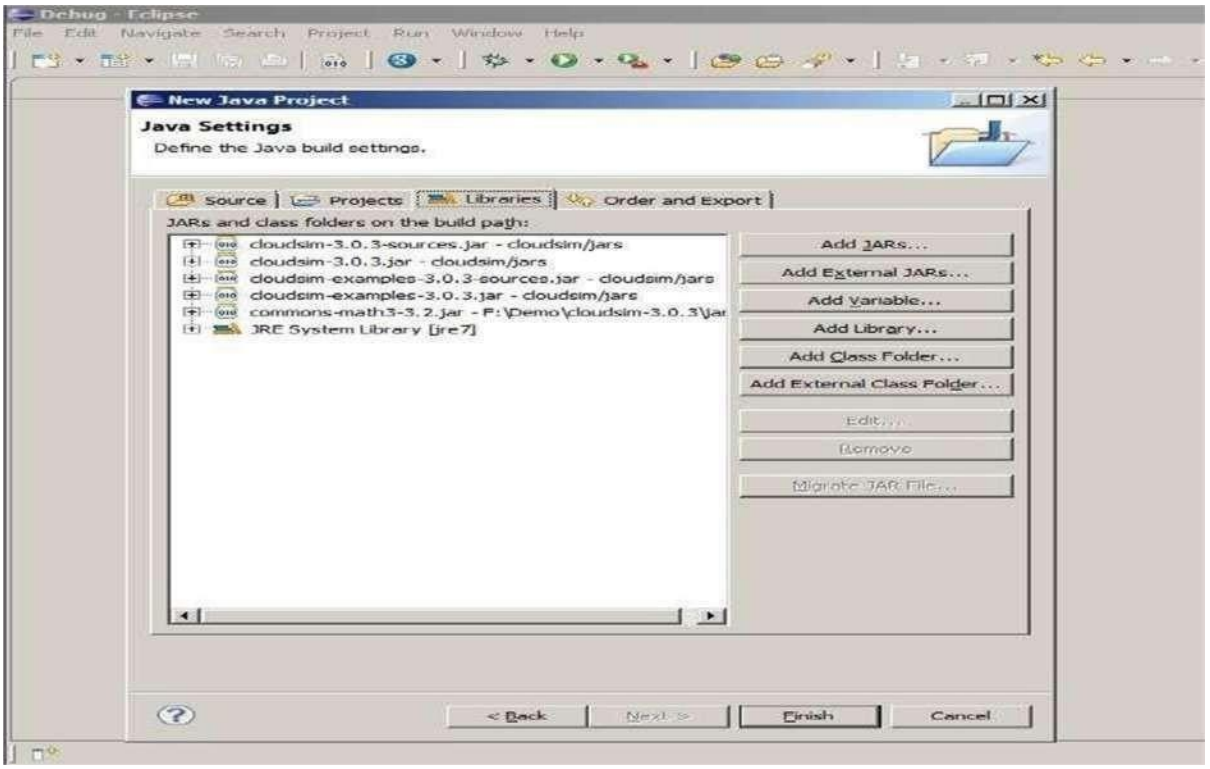
Step 12: Now open 'Libraries' tab and if you do not find commons-math3-3.x.jar (here 'x' means the minor version release of the library which could be 2 or greater) in the list then simply click on 'Add External Jar' (commons-math3-3.x.jar will be included in the project from this step)



Step 13: Once you have clicked on _Add External JAR's_ Open the path where you have unzipped the commons-math binaries and select _Commons-math3-3.x.jar' and click on open.

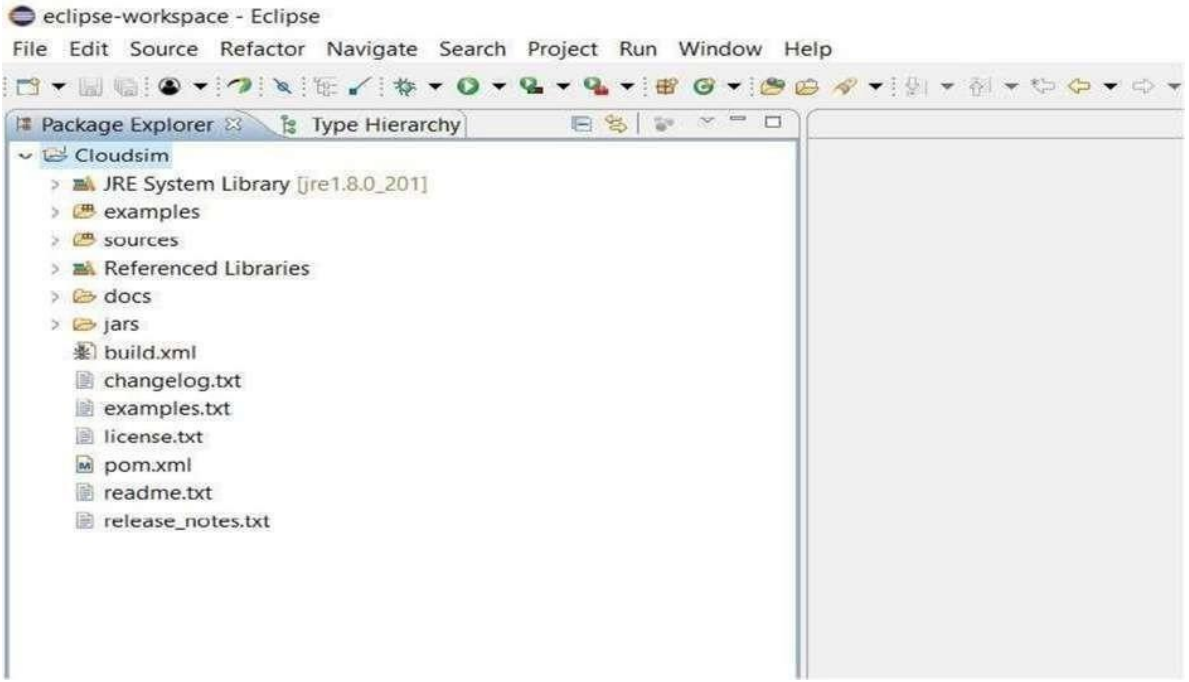


Step 14: Ensure external jar that you opened in the previous step is displayed in the list and then click on _Finish‘ (your system may take 2-3 minutes to configure the project)

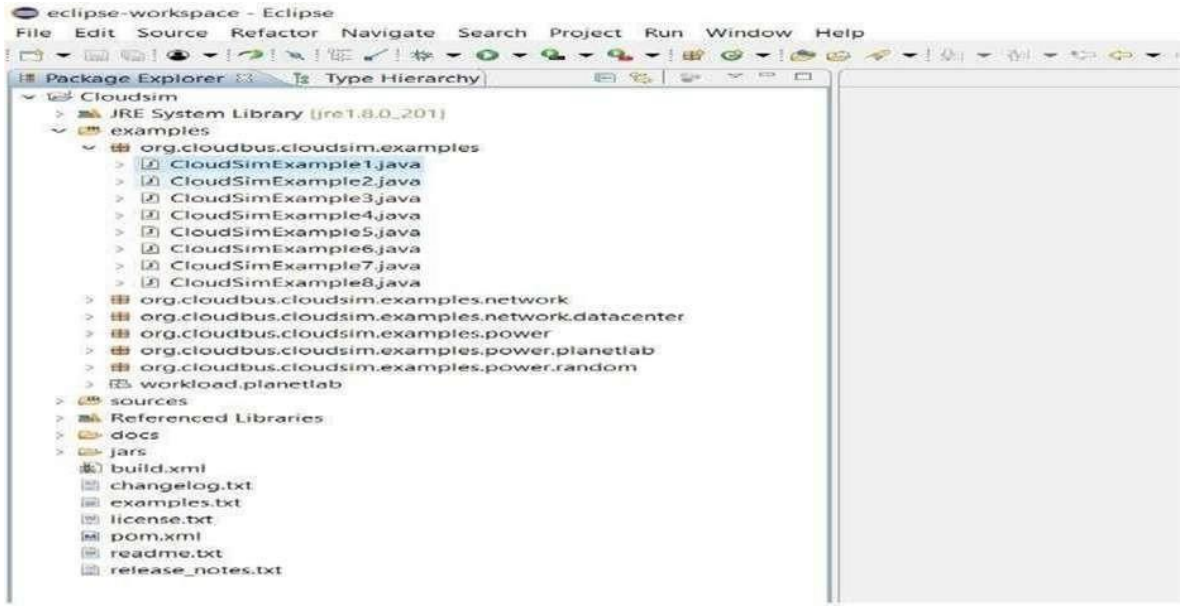


Step 15: Once the project is configured you can open the _Project Explorer_and start exploring the Cloudsim project. Also for the first time eclipse automatically start building the workspace for newly configured Cloudsim project, which may take some time depending on the configuration of the computer system.

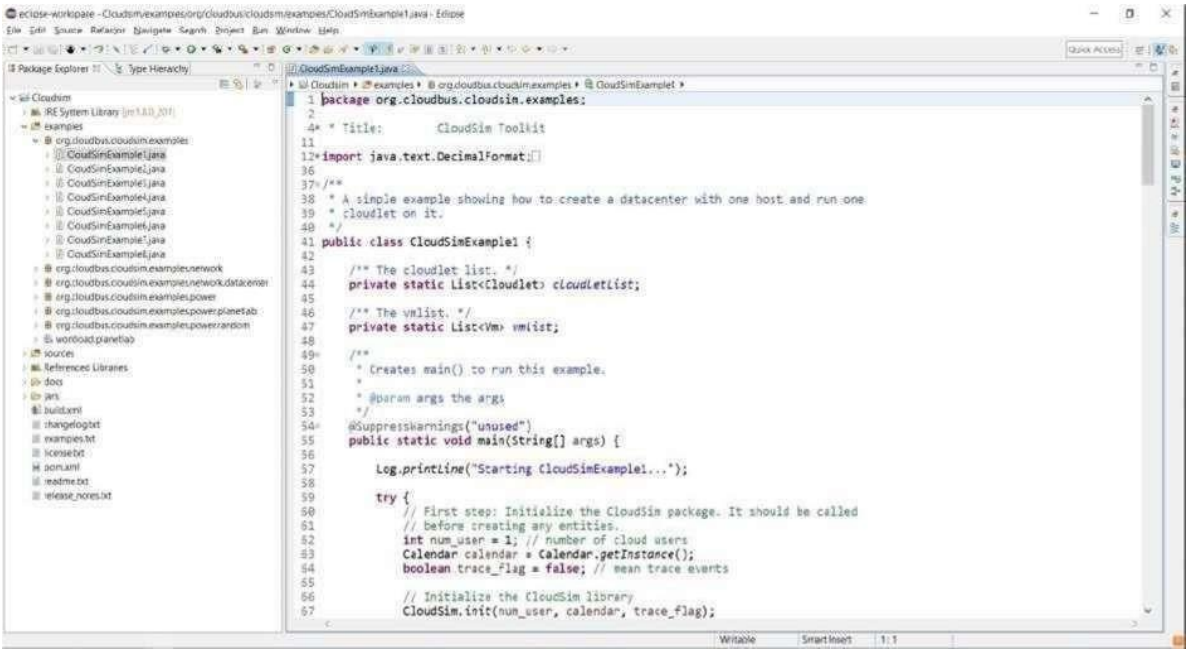
Following is the final screen which you will see after Cloudsim is configured.

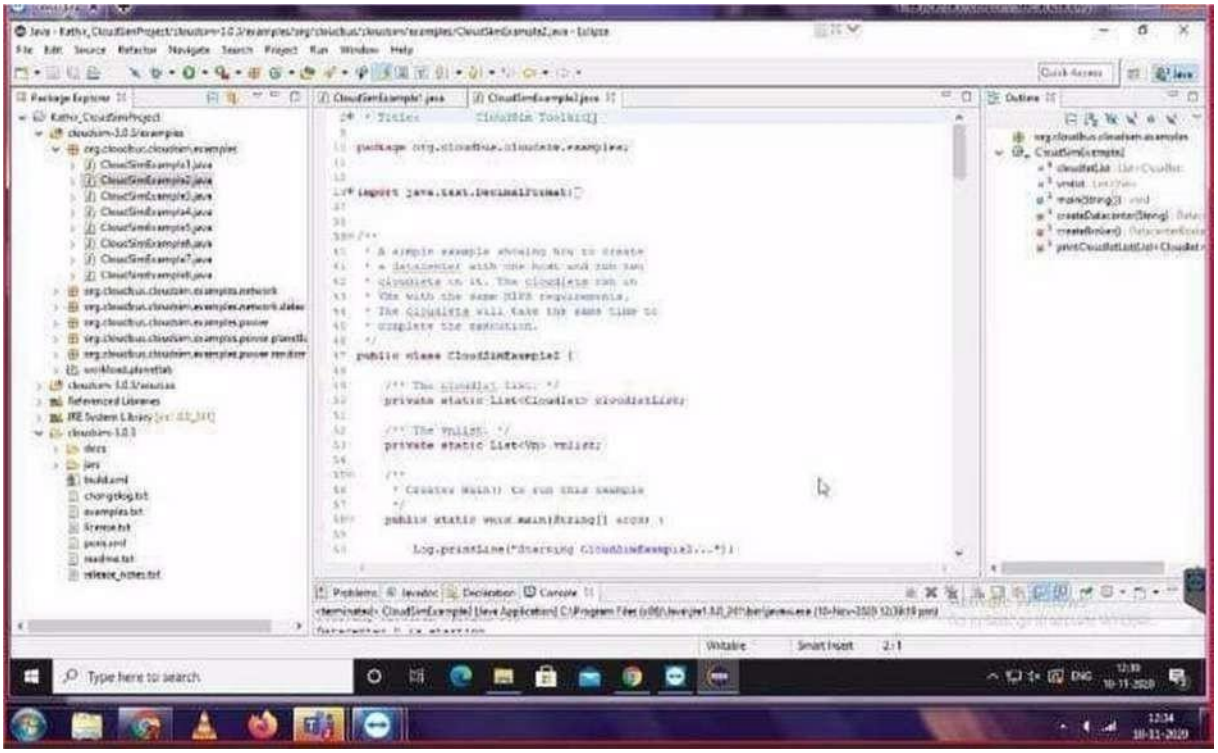


Step 16: Now just to check you within the **Project Explorer**, you should navigate to the **examples** folder, then expand the package org.cloudbus.cloudsim.examples and doubleclick to open the CloudsimExample1.java

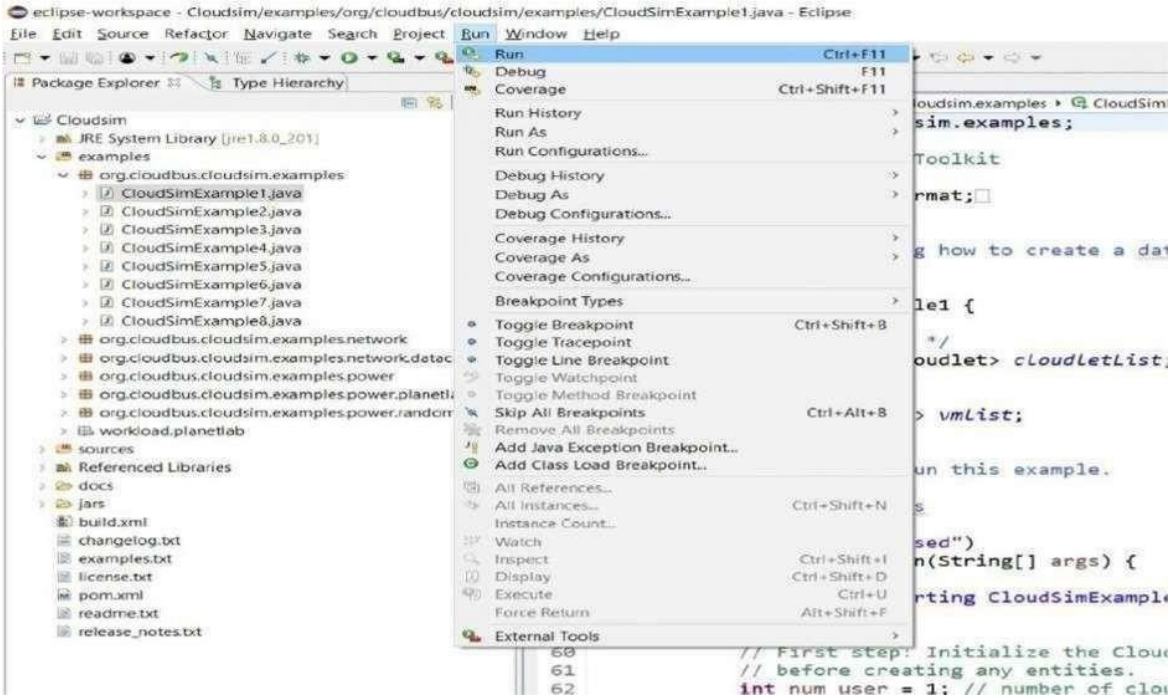


.

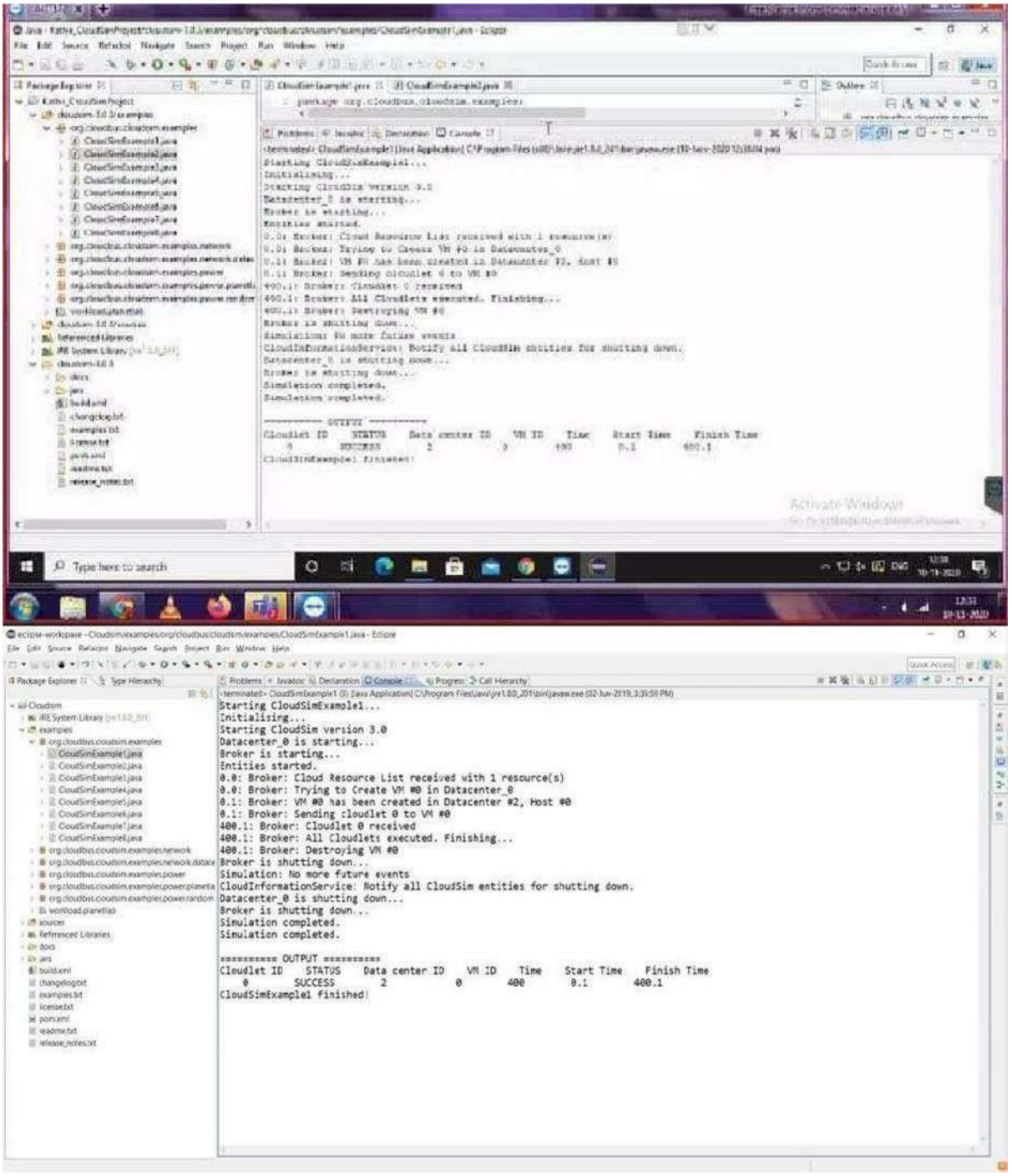




Step 17: Now navigate to the Eclipse menu Run ->Run_ or directly use a keyboard shortcut 'Ctrl + F11' to execute the CloudsimExample1.java.



Step 18: If it is successfully executed it should be displaying the following type to output inthe console window of the Eclipse IDE.



Result:

Thus the cloudsim is simulated using the Eclipse Environment successfully.

Ex.No:6 Date:	Find a procedure to transfer the files from one virtual machine to another virtual machine.

AIM:

To Find a procedure to transfer the files from one virtual machine to another virtual machine.

PROCEDURE

There are three ways to transfer the files from one virtual machine to another virtual machine

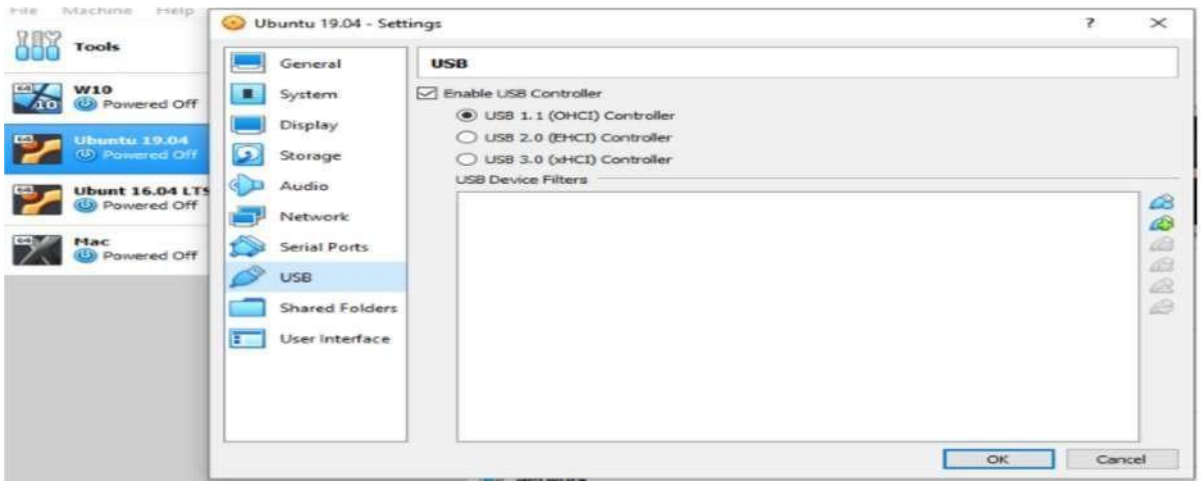
- Copy and paste
- USB drive
- Network share

STEP 1.Copy And Paste Data In VirtualBox

If you're using VirtualBox, with your virtual machine running, select Devices > Drag and Drop. Here, you can choose from Host to Guest, Guest to Host, and Bidirectional. There's also the default option, Disabled. For the best results, use Bidirectional.

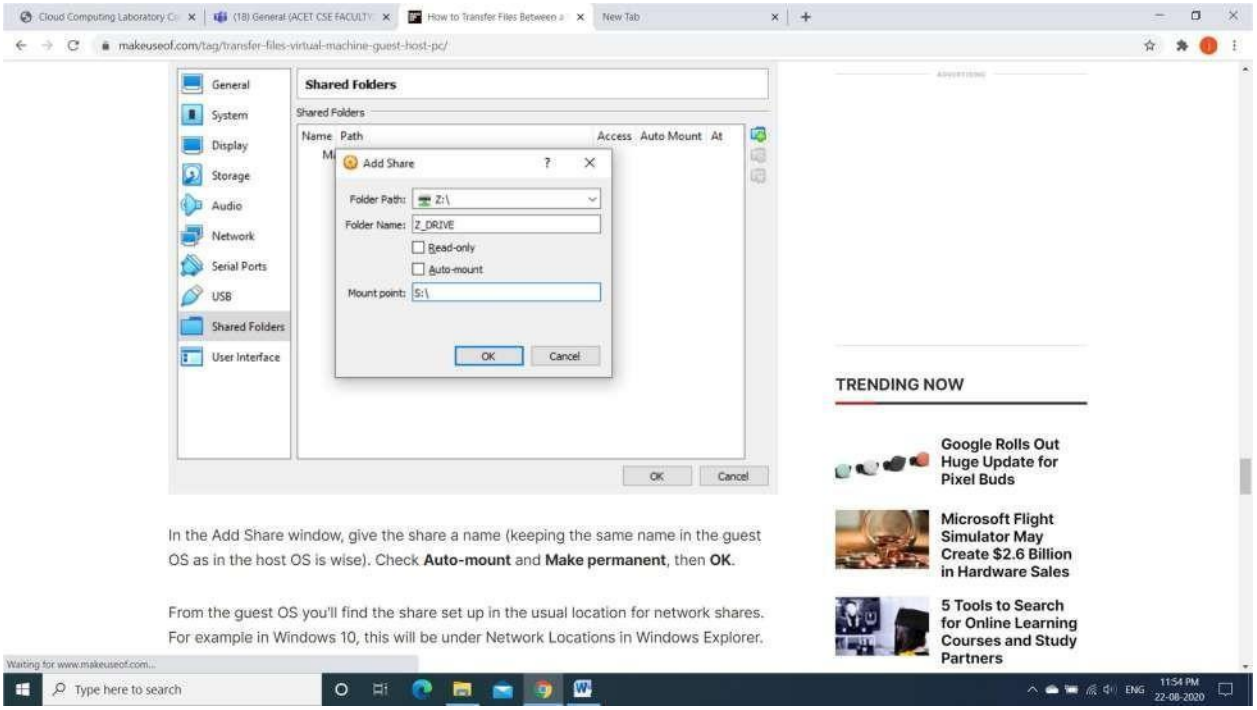
STEP 2:Share Files From A USB Stick In VirtualBox

- To be able to access USB devices from within VirtualBox, you'll need to enable USB access. For this, the VirtualBox Extension Pack is required from www.virtualbox.org/wiki/Downloads.
- Once you've done that, insert the USB device you wish to use. Next, open VirtualBox and click File > Preferences, then Extensions and click +. Browse to the downloaded Extension Pack, click Open, then when prompted, Install. Follow the prompts to complete the process. You can then check to confirm USB is enabled in Settings > USB



STEP 3. Create A Share In VirtualBox

- You should have already downloaded VirtualBox Guest Additions. This should be installed via Devices > Install Guest Additions, where you should browse for the appropriate EXE file. Follow the steps to the end, choosing the default options, then Finish.
- Launch VirtualBox and open Devices > Shared Folders > Shared Folders Settings. Click +, then in Folder Path click the arrow and select Other. Browse (the host OS) for the folder you're using as a share, highlight it, then Select Folder.in the Add Share window, give the share a name (keeping the same name in the guest OS as in the host OS is wise).
- Check Auto-mount and Make permanent, then OK.



RESULT:

Thus the procedure to transfer the files from one virtual machine to another virtual machine is executed successfully.

Ex.No: 7 Date:	**Install Hadoop single node cluster and run simple applications like wordcount.**

Aim:

To find the procedure to set up the one node Hadoop cluster.

1. HADOOP:

Apache Hadoop is an open-source software framework for storage and large-scale processing of data-sets on clusters of commodity hardware. Hadoop is an Apache top-level project being built and used by a global community of contributors and users. It is licensed under the Apache License 2.0.

The Apache Hadoop framework is composed of the following modules:

- **Hadoop Common** – contains libraries and utilities needed by other Hadoop modules
- **Hadoop Distributed** File System (HDFS) – a distributed file-system that stores data on commodity machines, providing very high aggregate bandwidth across the cluster.
- **Hadoop YARN** – a resource-management platform responsible for managing compute resources in clusters and using them for scheduling of users' applications.
- **Hadoop MapReduce** – a programming model for large scale data processing.

InstallationSteps:

Aim:
To find procedure to set up the one node Hadoop cluster.

Procedure:**Step 1:**

Installing Java is the main prerequisite for Hadoop. Install java1.7.

$sudo apt-get update
$sudo apt-get install openjdk-7-jdk
$sudo apt-get install openjdk-7-jre
$ java -version

java version "1.7.0_79"

OpenJDK Runtime Environment (IcedTea 2.5.6) (7u79-2.5.6-0ubuntu1.14.04.1)OpenJDK 64-Bit Server VM (build 24.79-b02, mixed mode)

Step 2:

SSH Server accepting password authentication (at least for the setup time).To install, run:
student@a4cse196:~$ suPassword:

root@a4cse196:/home/student# apt-get install openssh-server

Step 3:

Generate the ssh key
root@a4cse196:/home/student# ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa

Generating public/private rsa key pair.Created directory '/root/.ssh'.
Your identification has been saved in /root/.ssh/id_rsa. Your public key has been saved in /root/.ssh/id_rsa.pub.

The key fingerprint is: 77:a1:20:bb:db:95:6d:89:ce:44:25:32:b6:81:5d:d5 root@a4cse196The key's random art image is:
+--[RSA 2048] ---+

```
|  .... |
|  o .     E  |
|  o B . o    |
|  + * + .    |
|  . S + .    |
|  . o = .    |
|  . = +      |
|  o = .      |
|  . . o      |
```

+_____+

Step 4:

If the master also acts a slave (`ssh localhost` should work without a password)
root@a4cse196:/home/student# cat $HOME/.ssh/id_rsa.pub >>$HOME/.ssh/authorized_keysStep 5:
Create hadoop group and user:

Step 5.1 root@a4cse196:/home/student# **sudo addgroup hadoop**

Adding group `hadoop' (GID 1003) ...Done.
Step 5.2 root@a4cse196:/home/student# **sudo adduser --ingroup hadoop hadoop**

Adding user `hadoop' ...

Adding new user `hadoop' (1003) with group `hadoop' ...Creating home directory `/home/hadoop' ...
Copying files from `/etc/skel' ...Enter new UNIX password:

Retype new UNIX password:

passwd: password updated successfully Changing the user information for hadoop
Enter the new value, or press ENTER for the defaultFull Name []:
Room Number []:Work Phone []: Home Phone []: Other []:
Is the information correct? [Y/n] yroot@a4cse196:/home/student#

Step 6:

Copy your .tar file to home.(hadoop-2.7.0.tar.gz)

Step 7:

Extracting the tar file.

root@a4cse196:/home/student# **sudo tar -xzvf hadoop-2.7.0.tar.gz -C /usr/local/lib/**

Step 8:

Changing the Ownership

root@a4cse196:/home/student# **sudo chown -R hadoop:hadoop /usr/local/lib/hadoop-2.7.0**

Step 9:

Create HDFS directories:

root@a4cse196:/home/student# s**udo mkdir -p /var/lib/hadoop/hdfs/namenode**
root@a4cse196:/home/student# **sudo mkdir -p /var/lib/hadoop/hdfs/datanode**
root@a4cse196:/home/student# **sudo chown -R hadoop /var/lib/hadoop**

Step 10:

Check where your Java is installed: root@a4cse196:/home/student# **readlink -f /usr/bin/java**
/usr/lib/jvm/java-7-openjdk-amd64/jre/bin/java

Step 11:

Open gedit and do it root@a4cse196:/home/student# **gedit ~/.bashrcAdd to ~/.bashrc file:**
export JAVA_HOME=/usr/lib/jvm/java-7-openjdk-amd64export
HADOOP_INSTALL=/usr/local/lib/hadoop-2.7.0 export
PATH=$PATH:$HADOOP_INSTALL/bin
export PATH=$PATH:$HADOOP_INSTALL/sbin

export HADOOP_MAPRED_HOME=$HADOOP_INSTALL
export HADOOP_COMMON_HOME=$HADOOP_INSTALL
export HADOOP_HDFS_HOME=$HADOOP_INSTALL export
YARN_HOME=$HADOOP_INSTALL
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_INSTALL/lib/native
export HADOOP_OPTS="-Djava.library.path=$HADOOP_INSTALL/lib/native"

Step 12:

Reload source
root@a4cse196:/home/student# **source ~/.bashrc**

Step 13:

Modify JAVA_HOME in /usr/local/lib/hadoop-2.7.0/etc/hadoop/hadoop-env.sh:
root@a4cse196:/home/student# **cd /usr/local/lib/hadoop-2.7.0/etc/hadoop**
root@a4cse196:/usr/local/lib/hadoop-2.7.0/etc/hadoop# **gedit hadoop-env.sh export JAVA_HOME=${ JAVA_HOME}**
Changed this to below path

export JAVA_HOME=/usr/lib/jvm/java-7-openjdk-amd64

Step 14:

Modify /usr/local/lib/hadoop-2.7.0/etc/hadoop/core-site.xml to have something like:
root@a4cse196:/usr/local/lib/hadoop-2.7.0/etc/hadoop# **gedit core-site.xml**

```
<configuration>

<property>

<name>fs.default.name</name>

<value>hdfs://localhost:9000</value>

</property>

</configuration>
```

Step 15:

Modify /usr/local/lib/hadoop-2.7.0/etc/hadoop/yarn-site.xml to have something like:
root@a4cse196:/usr/local/lib/hadoop-2.7.0/etc/hadoop# **gedit yarn-site.xml**

```
<configuration>

<property>

<name>yarn.nodemanager.aux-services</name>
```

<value>mapreduce_shuffle</value>

</property>

<property>

<name>yarn.nodemanager.aux-services.mapreduce.shuffle.class</name>

<value>org.apache.hadoop.mapred.ShuffleHandler</value>

</property>

</configuration>

Step 16:

Create /usr/local/lib/hadoop-2.7.0/etc/hadoop/mapred-site.xml from template:
root@a4cse196:/usr/local/lib/hadoop-2.7.0/etc/hadoop# **cp /usr/local/lib/hadoop-2.7.0/etc/hadoop/mapred-site.xml.template /usr/local/lib/hadoop-2.7.0/etc/hadoop/mapred-site.xml**

Step 17:

Modify /usr/local/lib/hadoop-2.7.0/etc/hadoop/mapred-site.xml to have something like:
root@a4cse196:/usr/local/lib/hadoop-2.7.0/etc/hadoop# **gedit mapred-site.xml**

<configuration>

<property>

<name>mapreduce.framework.name</name>

<value>yarn</value>

</property>

</configuration>

Step 18:

Modify /usr/local/lib/hadoop-2.7.0/etc/hadoop/hdfs-site.xml to have something like:
root@a4cse196:/usr/local/lib/hadoop-2.7.0/etc/hadoop# **gedit hdfs-site.xml**
<configuration>

<property>

<name>dfs.replication</name>

<value>1</value>

</property>

<property>

<name>dfs.namenode.name.dir</name>

<value>file:/var/lib/hadoop/hdfs/namenode</value>

</property>

<property>

<name>dfs.datanode.data.dir</name>

<value>file:/var/lib/hadoop/hdfs/datanode</value>

</property>

</configuration>

Step 19:

Make changes in /etc/profile

$**gedit /etc/profile** JAVA_HOME=/usr/lib/jvm/java-7-openjdk-amd64
PATH=$PATH:$JAVA_HOME/bin
export JAVA_HOMEexport PATH
$**source /etc/profile**

Step 20:

root@a4cse196:/usr/local/lib/hadoop-2.7.0/etc/hadoop# **hdfs namenode –format**

Step 21:

Switch to hadoop user
start-dfs.sh

yes
yes
start-yarn.sh root@a4cse196:/home/hadoop# **jps**6334 SecondaryNameNode
6498 ResourceManager

6927 Jps

6142 DataNode

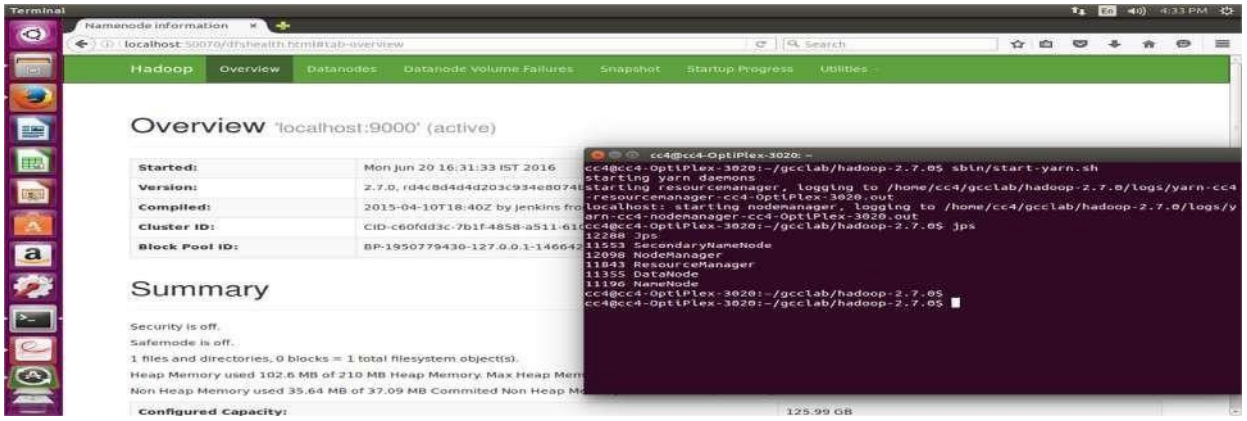
5990 NameNode

6696 NodeManager

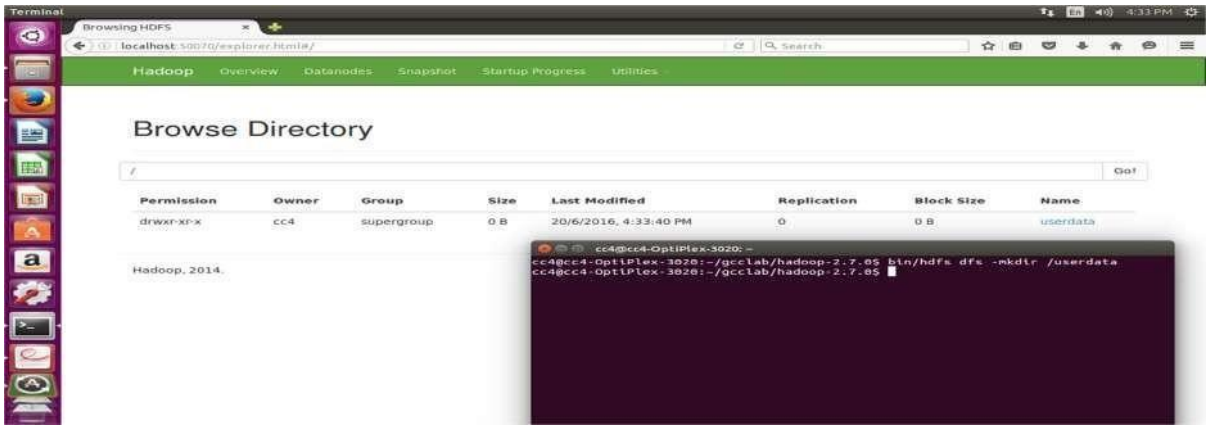
Step 22:

Browse the web interface for the **Name Node;** by default it is available at:

http://localhost:50070



Create the HDFS directories:



Mapreduce:

MapReduce is a programming model and an associated implementation for processing and generating large data sets with a parallel, distributed algorithm on a cluster. A MapReduce program is composed of a **Map()** procedure that performs filtering and sorting and a **Reduce()** method that performs a summary operation.

- **"Map" step**: Each worker node applies the "map()" function to the local data, and writes the output to a temporary storage. A master node ensures that only one copy of redundant input data is processed.
- **"Shuffle" step**: Worker nodes redistribute data based on the output keys (produced by the "map()" function), such that all data belonging to one key is located on the same worker node.
- **"Reduce" step:** Worker nodes now process each group of output data, per key, in parallel.

Steps:

Source Code:

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

  public static class TokenizerMapper
```

```
extends Mapper<Object, Text, Text, IntWritable>{

  private final static IntWritable one = new IntWritable(1);
  private Text word = new Text();

  public void map(Object key, Text value, Context context
                  ) throws IOException, InterruptedException {
    StringTokenizer itr = new StringTokenizer(value.toString());
    while (itr.hasMoreTokens()) {
      word.set(itr.nextToken());
      context.write(word, one);
    }
  }
}

public static class IntSumReducer
    extends Reducer<Text,IntWritable,Text,IntWritable> {
  private IntWritable result = new IntWritable();

  public void reduce(Text key, Iterable<IntWritable> values,
                     Context context
                     ) throws IOException, InterruptedException {
    int sum = 0;
    for (IntWritable val : values) {
      sum += val.get();
    }
    result.set(sum);
    context.write(key, result);
  }
}

public static void main(String[] args) throws Exception {
  Configuration conf = new Configuration();
  Job job = Job.getInstance(conf, "word count");
  job.setJarByClass(WordCount.class);
  job.setMapperClass(TokenizerMapper.class);
  job.setCombinerClass(IntSumReducer.class);
  job.setReducerClass(IntSumReducer.class);
  job.setOutputKeyClass(Text.class);
  job.setOutputValueClass(IntWritable.class);
  FileInputFormat.addInputPath(job, new Path(args[0]));
  FileOutputFormat.setOutputPath(job, new Path(args[1]));
  System.exit(job.waitForCompletion(true) ? 0 : 1);
```

```
  }
}
```

1. Set Environmental Variables:

```
export JAVA_HOME=/usr/java/default
export PATH=${JAVA_HOME}/bin:${PATH}
export HADOOP_CLASSPATH=${JAVA_HOME}/lib/tools.jar
```

2. Compile the source file to jar file,

```
$ bin/hadoop com.sun.tools.javac.Main WordCount.java
$ jar cf wc.jar WordCount*.class
```

3. Run the Application

```
$ bin/hadoop jar wc.jar WordCount /user/joe/wordcount/input /user/joe/wordcount/output
```

Output:

```
$ bin/hadoop fs -cat /user/joe/wordcount/output/part-r-00000`
Bye 1
Goodbye 1
Hadoop 2
Hello 2
World 2`
```

RESULT:

Thus the one node Hadoop cluster is installed and word count program to demonstrate the Map and Reduce task is done successfully.

Ex. No:8 **Creating and Executing Your First Container Using Docker.**

AIM:
To write a program to Creating and Executing Your First Container Using Docker.

PROCEDURE:

1. Install Docker on your machine
For Ubuntu:
First, update your packages:
$ sudo apt update
Next, install docker with apt-get:
$ sudo apt install docker.io
Finally, verify that Docker is installed correctly:
$ sudo docker run hello-world

- *For MacOSX:* you can follow this link.
- *For Windows:* you can follow this link.

2. Create your project
In order to create your first Docker application, I invite you to create a folder on your computer. It must contain the following two files:

- A '*main.py*' file (python file that will contain the code to be executed).
- A '*Dockerfile*' file (Docker file that will contain the necessary instructions to create the environment).

Normally you should have this folder architecture:
├── Dockerfile
└── main.py
0 directories, 2 files

3. Edit the Python file
You can add the following code to the '*main.py*' file:
#!/usr/bin/env python3
print("Docker is magic!")
Nothing exceptional, but once you see "*Docker is magic!*" displayed in your terminal you will know that your Docker is working.

3. Edit the Docker file
Some theory: the first thing to do when you want to create your Dockerfile is to ask yourself what you want to do. Our goal here is to launch Python code.

To do this, our Docker must contain all the dependencies necessary to launch Python. A linux (Ubuntu) with Python installed on it should be enough.

The first step to take when you create a Docker file is to access the DockerHub website. This site contains many pre-designed images to save your time (for example: all images for linux

or code languages). In our case, we will type ‘Python’ in the search bar. The first result is the official image created to execute Python. Perfect, we’ll use it!

DOCKERFILE:

```
# A dockerfile must always start by importing the base image.
# We use the keyword 'FROM' to do that.
# In our example, we want import the python image.
# So we write 'python' for the image name and 'latest' for the version.
FROM python:latest
# In order to launch our python code, we must import it into our image.
# We use the keyword 'COPY' to do that.
# The first parameter 'main.py' is the name of the file on the host.
# The second parameter '/' is the path where to put the file on the image.
# Here we put the file at the image root folder.
COPY main.py /
# We need to define the command to launch when we are going to run the image.
# We use the keyword 'CMD' to do that.
# The following command will execute "python ./main.py".
CMD [ "python", "./main.py" ]
```

4. Create the Docker image

Once your code is ready and the Dockerfile is written, all you have to do is create your image to contain your application.
$ docker build -t python-test .
The ’*-t*’ option allows you to define the name of your image. In our case we have chosen ’*python-test*’ but you can put what you want.

5. Run the Docker image
Once the image is created, your code is ready to be launched.

$ docker run python-test

You need to put the name of your image after ‘*docker run*’.
There you go, that’s it. You should normally see “Docker is magic!” displayed in your terminal.

Useful commands for Docker
A list of commands that may be useful to you on Docker.

- List your images.

$ docker image ls

- Delete a specific image.

$ docker image rm [image name]

- Delete all existing images.

$ docker image rm $(docker images -a -q)

- List all existing containers (running and not running).

$ docker ps -a

- Stop a specific container.

$ docker stop [container name]

- Stop all running containers.

$ docker stop $(docker ps -a -q)

- Delete a specific container (only if stopped).

$ docker rm [container name]

- Delete all containers (only if stopped).

$ docker rm $(docker ps -a -q)

- Display logs of a container.

$ docker logs [container name]

To Run:

The project launch is in two parts: build and start.

Build

```
$ docker build -t [image_name] .
```

Launch

```
$ docker run [image_name]
```

Result:

Thus, the Creating and Executing Your First Container Using Docker executed and verified successfully.

Ex. No:9 ***Run a Container from Docker Hub***

AIM:
To write a program to run a container from Docker hub.

PROCEDURE:
Run a container from docker hub Run docker -
h,
$ docker -h
Flag shorthand -h has been deprecated, please use --help Usage:

docker [OPTIONS] COMMAND

A self-sufficient runtime for containers
...
Management Commands: builder
Manage builds config Manage
Docker configs container Manage
containers engine
Manage the docker engine
image Manage images network
Manage networks node
Manage Swarm nodes plugin
Manage plugins secretManage
Docker secrets service Manage
services stack Manage Docker stacks
swarm Manage Swarm system
Manage Docker trust Manage trust
on Docker images volume Manage
volumes

The Docker command line can be used to manage several features of the Docker Engine. In this lab, we will mainly focus on the container command.

If podman is installed, you can run the alternative command for comparison. sudo podman -h
You can additionally review the version of your Docker installation, docker

Version
Client:
Version: 19.03.6
...
Server: Docker Engine - Community
Engine
Version: 19.03.5
...

sudo podman version --events-backend=none
Version: 2.1.1
API Version: 2.0.0
Go Version: go1.15.2

Built: Thu Jan 1 00:00:00 1970 OS/Arch:
linux/amd64

Step 1: Run your first container

We are going to use the Docker CLI to run our first container. Open a terminal on your local computer Run docker container run -t ubuntu
top

Use the docker container run command to run a container with the ubuntu image using the top command. The -t flags allocate a pseudo-TTY which we need for the top to work correctly.
$ docker container run -it ubuntu top Unable to
find image 'ubuntu:latest' locally latest: Pulling
from library/ubuntu aafe6b5e13de:
Pull complete 0a2b43a72660: Pull complete 18bdd1e546d2:
Pull complete 8198342c3e05:
Pull complete f56970a44fd4: Pull complete
Digest:sha256:f3a61450ae43896c4332bda5e78b453f4a93179045f20c8181043b26b
5e79028 Status: Downloaded newer image for ubuntu:latest

The docker run command will result first in a docker pull to download the ubuntu image onto your host. Once it is downloaded, it will start the container. The output for the running container should look like this:

top - 20:32:46 up 3 days, 17:40, 0 users, load average: 0.00, 0.01, 0.00 Tasks: 1 total, 1 running, 0 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.1 sy, 0.0 ni, 99.9 id, 0.0 wa, 0.0

KiB Mem : 2046768 total, 173308 free, 117248 used, 1756212 buff/cache
KiB Swap: 1048572 total, 1048572 free, 0 used. 1548356 avail Mem
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+
COMMAND 1 root 20 0 36636 3072 2640 R 0.3 0.2 0:00.04 top

Inspect the container with docker container exec
The docker container exec command is a way to "enter" a running container's namespaces with a new process.

Open a new terminal, select Terminal > New Terminal.

Using play-with-docker.com, to open a new terminal connected to node1, click "Add New Instance" on the lefthand side, then ssh from node2 into node1 using the IP that is listed by 'node1 '. For example:

[node2] (local) root@192.168.0.17 ~
$ ssh 192.168.0.18
[node1] (local) root@192.168.0.18 ~ $
In the new terminal, use the docker container ls command to get the ID of the running container you just created.

$ docker container ls			
CONTAINER ID IMAGE COMMAND NAMES	CREATED	STATUS	PORTS
b3ad2a23fab3 ubuntu "top" goofy_nobel	29 minutes ago	Up 29 minutes	

$ docker container exec -it b3ad2a23fab3 bash
root@b3ad2a23fab3:/#

The docker container exec command to "enter" our container's namespaces with our bash process. Using docker container exec with bash is a common pattern to inspect a docker container.
Notice the change in the prefix of your terminal. e.g. root@b3ad2a23fab3:/. This is an indication that we are running bash "inside" of our container.
From the same terminal, run ps -ef to inspect the running processes.

root@b3ad2a23fab3:/# ps -ef
UID PID PPID C STIME TTY TIME CMD
root 1 0 0 20:34 ? 00:00:00 top root
17 0 0 21:06 ? 00:00:00 bash root 27
17 0 21:14 ? 00:00:00 ps -ef
You should see only the top process, bash process and our ps process.
root@b3ad2a23fab3:/# exit
exit $ ps -ef
Lots of processes! docker ps -
a docker rm <CONTAINER ID>

Step 2: Run Multiple Containers
Explore the Docker Hub
The Docker Hub is the public central registry for Docker images, which contains community and official images.
Run an Nginx server

Let's run a container using the official Nginx image from the Docker Hub.
$ docker container run --detach --publish 8080:80 --name nginx nginx
Unable to find image 'nginx:latest' locally latest:
Pulling from library/nginx
36a46ebd5019: Pull complete 57168433389f:
Pull complete 332ec8285c50: Pull complete

Digest:
sha256:c15f1fb8fd55c60c72f940a76da76a5fccce2fefa0dd9b17967b9e40b0355316
Status: Downloaded newer image for nginx:latest
5e1bf0e6b926bd73a66f98b3cbe23d04189c16a43d55dd46b8486359f6fdf048 Nginx is a lightweight web server. You can access it on port 8080 on your localhost. Access the nginx server on localhost:8080. curl localhost:8080 will return the HTML home page of Nginx,

<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style> body
{ width: 35em; margin:
0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style> </head>
<body>
<h1>Welcome to nginx!</h1>

If you are using play-with-docker, look for the 8080 link near the top of the page, or if you run a Docker client with access to a local browser,

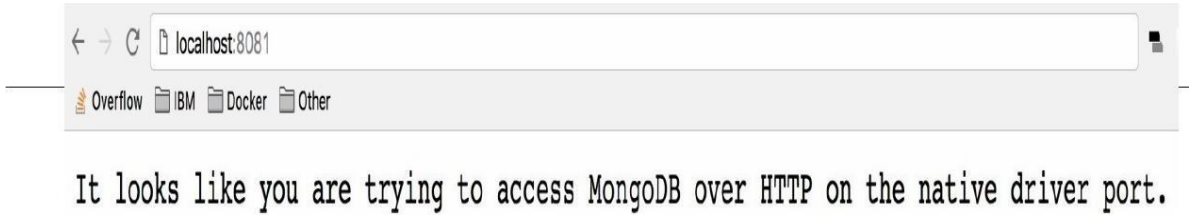
Run a mongo DB server

Now, run a mongoDB server. We will use the official mongoDB image from the Docker Hub. Instead of using the latest tag (which is the default if no tag is specified), we will use a specific version of the mongo image.

$ docker container run --detach --publish 8081:27017 --name mongo mongo:4.4
Unable to
find image mongo:4.4 locally 4.4: Pulling from library/mongo d13d02fa248d: Already
exists bc8e2652ce92: Pull complete 3cc856886986: Pull complete c319e9ec4517: Pull
complete b4cbf8808f94: Pull complete cb98a53e6676: Pull complete f0485050cd8a: Pull
complete ac36cdc414b3: Pull complete 61814e3c487b: Pull complete 523a9f1da6b9: Pull complete 3b4beaef77a2:
Pull complete
Digest:sha256:d13c897516e497e898c229e2467f4953314b63e48d4990d3215d876ef9d1fc7c Status: Downloaded newer image for mongo:4.4
d8f614a4969fb1229f538e171850512f10f490cb1a96fca27e4aa89ac082eba5

Access localhost:8081 to see some output from mongo. curl localhost:8081
which will return a warning from MongoDB,

It looks like you are trying to access MongoDB over HTTP on the native driver port. If you are
using play-with-docker, look for the 8080 link near the top of the page.



Check your running containers with docker container ls

$ docker container ls

CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES

d6777df89fea nginx "nginx -g 'daemon ..." Less than a second ago Up 2 seconds 0.0.0.0:8080-

>80/tcp nginx

ead80a0db505 mongo "docker-entrypoint..." 17 seconds ago Up 19 seconds 0.0.0.0:8081->27017/tcp mongo af549dccd5cf ubuntu "top" 5 minutes ago Up 5 minutes priceless_kepler

Step 3: Clean Up

First get a list of the containers running using docker container ls.

$ docker container ls

CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES

d6777df89fea nginx "nginx -g 'daemon ..." 3 minutes ago Up 3 minutes 0.0.0.0:8080-

>80/tcp nginx

ead80a0db505 mongo "docker-entrypoint..." 3 minutes ago

Up 3 minutes 0.0.0.0:8081->27017/tcp mongo

af549dccd5cf ubuntu "top" 8 minutes ago

Up 8 minutes priceless_kepler

Next, run docker container stop [container id] for each container in the list. You can also use the names of the containers that you specified before.

$ docker container stop

d67 ead af5 d67 e a

d

af

5

1. *Remove the stopped containers*

docker system prune is a really handy command to clean up your system. It will remove any stopped

containers, unused volumes and networks, and dangling images.

$ docker system prune

WARNING!

This will remove:

- all stopped containers
- all volumes not used by at least one container
- all networks not used by at least one container
- all dangling images Are you sure you want to continue? [y/N] y Deleted Containers:

7872fd96ea4695795c41150a06067d605f69702dbcb9ce49492c9029f0e1b44b
60abd5ee65b1e2732ddc02b971a86e22de1c1c446dab165462a08b037ef7835c
31617fdd8e5f584c51ce182757e24a1c9620257027665c20be75aa3ab6591740
Total reclaimed space: 12B

Result:

Thus, the write program to run a container from Docker Hub executed and verified successfully.